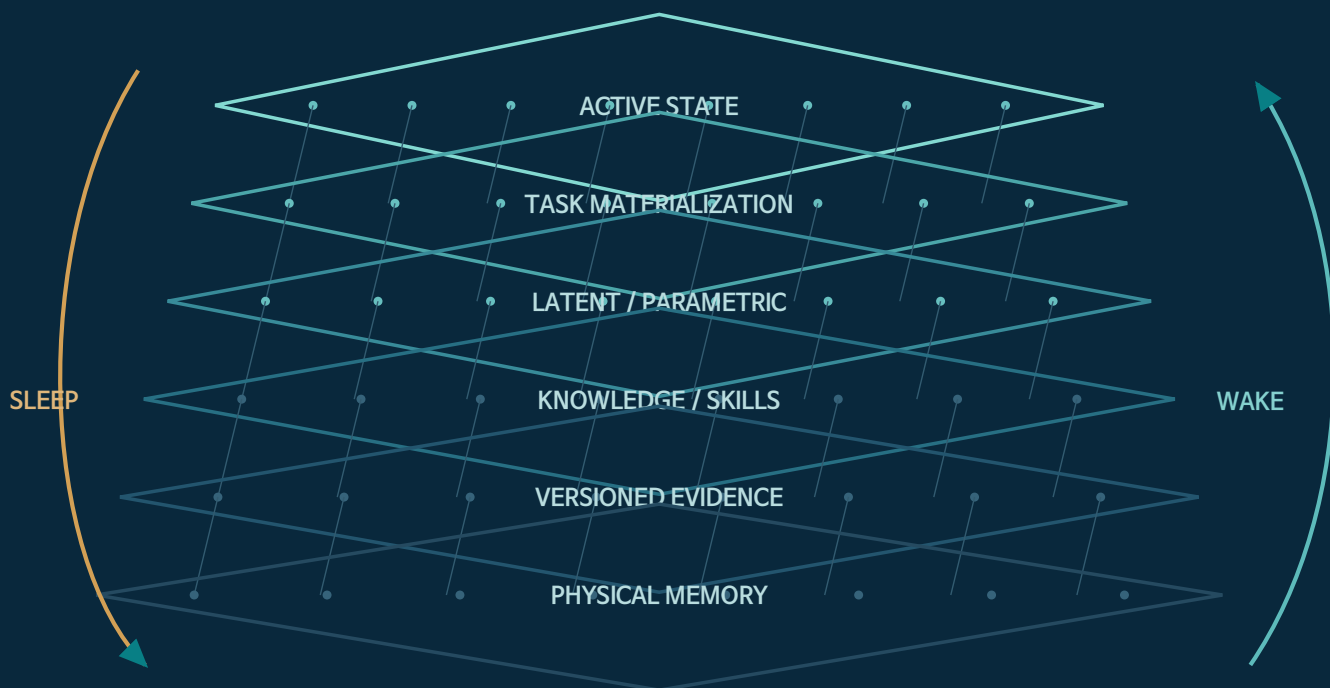


SLEEP-TIME COMPUTE & NEURAL MEMORY

경험을 지속 가능한
실행 상태로 바꾸는 AI

Agent memory · Continual learning · HOPE / CMS
HBM / HBF hardware co-design



EXECUTIVE BRIEF

핵심 판단

재사용 가능한 상태는 자산이다. 그러나 검증·갱신·이동 비용까지 포함할 때만 자산으로 남는다.

Sleep-time compute의 핵심은 더 오래 생각하는 것 자체가 아니다. 이미 지불한 추론·탐색 비용을 **다음 과제에서 재사용할 수 있는 상태로 바꾸고**, 그 상태의 유효성과 비용을 지속해서 관리하는 데 있다. 최초 연구는 질문 이전 계산을 미래 질의에 상각했고, 이후 연구는 latent state, skill, reusable KV, memory program으로 변환 대상을 확장했다. [01], [09], [28], [36], [33]

01

저장과 실행을 분리

canonical evidence와 runtime artifact는 서로 다른 생애주기를 가진다.

02

검증을 승격의 조건으로

빈도·관련성보다 전이 효율과 회귀 위험을 측정한다.

03

물리 배치는 별도 최적화

update frequency와 read bandwidth를 혼동하지 않는다.

연구가 보여주는 것

메모리의 존재, 현재 사실의 유효성, 실행 시 활성화, 판단에 미치는 영향은 별개다. 과거 사실을 정확히 회수해도 최신 상태를 잘못 판단할 수 있으며, 올바른 기억도 필요 이상으로 사용될 수 있다. [18], [13]

계층의 다음 단계는 단순 요약 저장소가 아니다. 경험에서 모델별 adapter를 생성하거나, 환경을 예측하는 프로그램을 만들고, 그 프로그램을 다음 실행에 사용하는 접근이 등장했다. [10], [34]

우리가 제안하는 것

원본 근거와 파생 artifact를 분리한 **versioned state substrate** 위에, sleep compiler와 독립 검증기를 둔다. cold storage에서 읽어 온 후보를 hot working set으로 materialize하고, 실제 미래 효율이 확인된 버전만 활성화한다.

다만 이 설계는 문헌을 연결한 가설이다. slow CMS를 HBF에 일괄 배치하거나, memory 수가 늘면 성능이 자동으로 증가한다고 전제하지 않는다.

의사결정 요약

지금 우선 투자할 대상은 거대한 dreaming model보다 **근거·버전·효용 계측이 가능한 학습/배치 인터페이스**다. 이후 같은 예산에서 retrieval, latent memory, LoRA, skill compilation을 비교해야 한다.

READER'S GUIDE

보고서 안내

주간 뉴스 목록 대신, 연구 메커니즘과 시스템 설계의 연결을 따라 읽는다.

01	개념 지도와 문헌 읽는 법	4
02	Sleep-time compute와 HOPE / CMS	7
03	Recurrent·latent·parametric memory	10
04	Consolidation·credit·influence	12
05	Dreaming과 self-improving agents	16
06	Validity·transaction·security	21
07	Serving·KV·HBM / HBF	24
08	통합 아키텍처와 실험 계획	31
09	산업 신호·정정·추적 목록	36
10	주석형 참고문헌 및 제작 정보	41

독자별 추천 경로

표 01. 목적에 따른 읽기 경로

독자 / 목적	우선 읽을 질문
연구 기획	어떤 상태를 학습할 것인가? 신경망 내부의 adaptation과 외부 memory control은 어떻게 다른가?
HW / 시스템 설계	논리적 갱신 주기와 실제 read/write traffic은 어떻게 연결되는가? HBF의 손익 분기점은 무엇인가?
Agent / 제품 개발	무엇을 보존하고 언제 무효화할 것인가? 복구·권한·평가를 어디에서 강제할 것인가?

원문 결과 **설계 가설** **제품 신호** 세 종류를 구분한다. 숫자는 본문과 figure caption에 범위·분모·비교군을 함께 표기했다. 참고문헌 번호를 누르면 주석형 출처 목록으로 이동한다.

01 / SCOPE & EVIDENCE

근거를 읽는 법

확인한 사실, 저자 주장, 우리의 설계 추론을 서로 다른 층에 둔다.

이 보고서는 이전 대화의 주간 조사와 GitHub 초안을 출발점으로, 핵심 논문의 제목·식별자·제출일·초록 및 일부 본문/도판을 다시 대조한 **근거 등급형 기술 종합**이다. 전체 분야를 빠짐없이 검색한 systematic review나 독립 재현 연구를 주장하지 않는다. 주요 조사 창은 2026년 8월~10월 초이며 2024~2025년 기반 연구를 연결했다.

표 02. 근거 유형과 사용 범위

표기	포함 자료	해석 범위
P / 논문	arXiv preprint 및 확인된 학술지 논문	실험은 저자 보고값. 채택·peer review·재현 여부를 자동 추정하지 않는다.
D / 공식 문서	연구 블로그·제품 문서·기업 발표	알고리즘·제품 기능·표준화 사실 확인. 성능 우위를 독립 검증한 근거는 아니다.
C / 커뮤니티	직접 읽은 사용자 게시물	문제 발굴 신호. 표본·시장 점유율·일반적 성능으로 외삽하지 않는다.
H / 가설	문헌을 연결한 본 보고서 제안	구현 결과가 아니며 실험으로 반증할 수 있도록 설계한다.

검토 규칙

동일 arXiv ID의 버전이 달라질 때는 결과를 합치지 않는다. 예를 들어 PlanFence는 본문에서 v1에 고정했다. 최초 제출일과 업데이트일, 공개 문서 확인일을 구분하며, 웹페이지가 없는 경우 제목만으로 수치를 복원하지 않았다. [22]

특히 HOPE/CMS는 원문 식과 실제 도식을 확인했다. HBFSim은 Figure 1 및 thermal/refresh 절을 확인하고, 원문의 시뮬레이션 실행 시간 개선과 LLM inference 성능을 구별했다. [03], [46]

매체 제작 규칙

도식은 논문의 의미를 다시 설계한 **독립 벡터 그림**이다. 원문 figure를 무단 복제하거나 장식용 AI 이미지를 실험 결과처럼 사용하지 않았다. 수치 그래프는 확인한 저자 보고값만 재시각화하고 출처를 붙였다.

서지 대장은 55개 출처를 포함한다. 이전 조사에서 추가 언급됐으나 이번 판에서 충분히 재검증하지 않은 항목은 추적 목록으로 남겼으며, 핵심 결론의 증거 수에는 포함하지 않았다.

해석의 경계

“Compiled experience scaling”은 유망한 연구 관점이지, 현재 입증된 보편적 scaling law가 아니다. “학술지 수준 구성”은 편집·근거 표현의 목표이며 이 PDF가 학술지 심사를 받았다는 뜻이 아니다.

01 / CONCEPT MAP

Memory를 네 축으로 분해하기

무엇을 기억하는가, 어떻게 표현하는가, 언제 갱신하는가, 어디서 실행하는가는 서로 다른 질문이다.

MEMORY IS NOT ONE THING



CROSS-CUTTING CONTROL

version / dependency / authority / activation / influence

Placement: SRAM · HBM · DRAM/CXL · HBF · SSD

그림 01. 기억의 내용·표현·실행 형태와 제어 메타데이터를 분리한 분석 지도. 본 보고서 종합 도식. [18], [09], [36]

표현과 생애주기를 섞지 않는다

episodic memory도 오래 보존할 수 있고, semantic rule도 곧바로 무효화될 수 있다. KV가 빠르게 변한다고 항상 폐기 가능하지도 않다. 한 representation이 다른 representation보다 본질적으로 더 “장기”라고 단정하면 정책이 잘못된다.

graph나 vector는 접근 방법이다. 반면 권한, 유효 기간, generation은 무엇을 근거로 실행할 수 있는지를 결정한다. 데이터 모델과 신뢰 모델은 별개다.

읽기 경로와 학습 경로를 분리한다

retrieval은 기존 객체를 선택하고, recurrent update는 현재 상태를 바꾸며, parametric learning은 실행 함수의 파라미터를 바꾼다. 세 가지는 모두 memory 효과를 내지만 비용·가역성·관측 가능성이 다르다. [08], [05]

제품 설계에서 “memory API 하나”로 감추기 전에, 각 연산의 입력 근거와 갱신 대상, 만료 조건을 명시해야 한다.

용어의 핵심

본 보고서에서 **canonical**은 변경 불가능하거나 진실이라는 뜻이 아니라, 출처·버전 계약 아래 관리되는 기준 상태라는 뜻이다. 신뢰할 수 있는 원본도 새 증거로 반박될 수 있다.

01 / RESEARCH LANDSCAPE

기술 계보: 적응에서 상태 관리로

빠른 적응, 외부 기억, 배경 계산이 하나의 lifetime system으로 만나고 있다.



그림 02. 대표 자료의 공개 순서. 분야가 단일 선형 단계로 발전했다는 증명은 아니며, 병렬 연구 흐름을 묶은 지도다. 발표 시점 기반 재구성.
[08], [07], [01], [03], [51], [28], [09], [32]

연속성

TTT는 hidden state를 학습 모델로 보는 길을 열었고, Titans와 Nested Learning은 memory update와 모델 구조를 함께 다룬다. 2026년 agent 연구는 별도의 외부 state, skill, program까지 갱신 대상으로 넓힌다. [08], [07], [03]

단절을 과장하지 않는다

“이번 주부터 제어가 중요해졌다”는 식의 서사는 검색 창에 민감하다. 이 보고서는 뉴스 빈도보다 메커니즘의 연결과 재현 가능한 비교 문제에 무게를 둔다. 각 접근은 서로 대체되기보다 다른 계층에서 공존할 수 있다.

02 / SLEEP COMPUTE

Sleep-time compute의 경제학

추론을 밖으로 옮기는 것과 지식을 학습하는 것은 다르다. 둘 다 비용표에 올려야 한다.

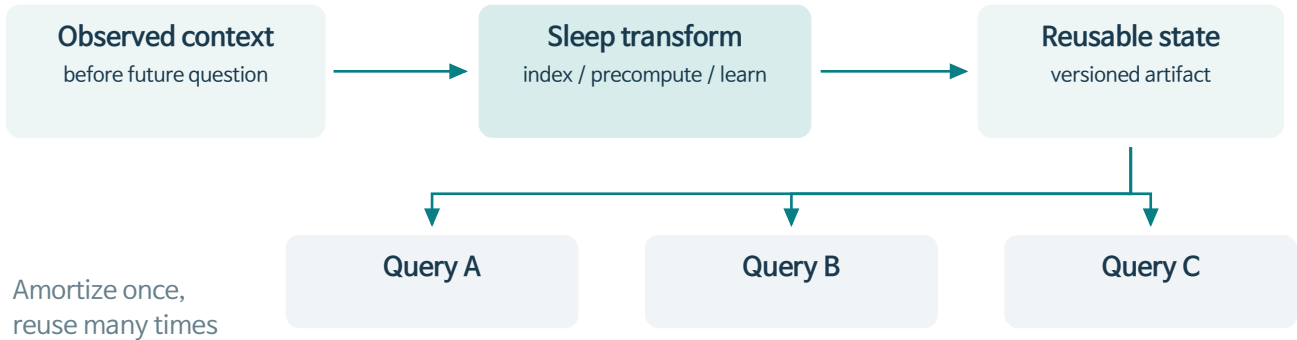


그림 03. 질문 이전 계산과 질문 이후 계산의 분리. 재사용 대상과 미래 query의 예측 가능성이 비용 상각을 결정한다. 개념 재구성. [01], [02]

원 연구의 기여

Sleep-time Compute는 context가 주어졌지만 아직 미래 질문이 확정되지 않은 구간에서 계산을 수행한다. stateful 추론 실험에서는 동일 정확도에 필요한 test-time compute를 약 5배 줄였고, multi-query 설정에서는 평균 query cost를 2.5배 낮췄다고 보고한다. 이 수치는 모든 agent workload에 대한 보장이 아니다. [01]

유용한 조건과 실패 조건

자주 반복되는 질의 분포, 안정적인 context, 비싼 재추론이 있다면 사전 계산의 가치가 커진다. 반대로 거의 사용되지 않는 자료를 깊게 요약하거나 세계가 빠르게 바뀌면, sleep compute와 invalidation 비용을 회수하지 못한다.

$$N \cdot \Delta C_{\text{query}} \gg C_{\text{sleep}} + C_{\text{refresh}} + C_{\text{storage / transfer}}$$

비용 단위를 통일한 단순 손익 조건. 본 보고서의 분석식이며 경험 법칙이 아니다.

Background maintenance ≠ learning

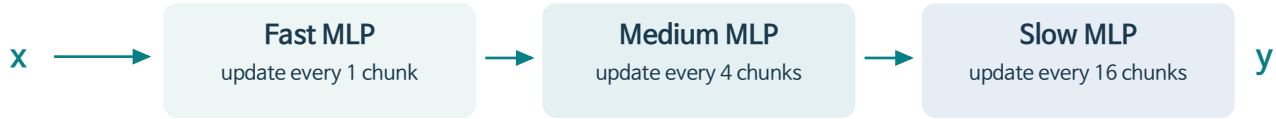
DMA, compaction, GC는 자원 효율을 높일 수 있지만 그것만으로 agent가 학습했다고 볼 수 없다. 현재 요청 비용 절감과 미래 과제 능력 변화를 분리해서 측정한다.

02 / FOUNDATIONAL MECHANISM

HOPE / Nested Learning을 정확히 읽기

학습 주기의 계층은 저장 장치의 계층과 닮았지만, 동일하지 않다.

FORWARD: EVERY TOKEN MAY READ THE ENTIRE CHAIN



UPDATE: DIFFERENT SCHEDULES



Illustrative chunk schedule; not the paper's measured timing

그림 04. CMS의 read/forward와 update schedule은 다르다. 아래 1/4/16 주기는 설명용이며 원문 실험 설정을 재현한 값이 아니다. 원문 Eq.70 - 71 기반 독립 재구성. [03]

원문의 CMS

Nested Learning은 서로 다른 빈도로 적응하는 연결된 optimization problem을 강조한다. CMS는 서로 다른 update frequency를 갖는 MLP 블록의 연쇄로 정의된다. HOPE는 self-modifying Titans의 적응성과 CMS의 지속적 저장을 결합한다. [03], [04]

Agent 시스템으로 확장할 때

episodic store→skill→LoRA라는 데이터 변환 계층은 유용하지만, 그것이 곧 원문의 CMS는 아니다. 외부 state의 lifecycle을 여러 timescale로 관리하는 설계는 CMS에서 영감을 얻은 시스템 가설로 불러야 한다.

가장 중요한 정정

느리게 갱신된다고 드물게 읽히는 것은 아니다. Eq.70의 forward chain에 참여하는 느린 블록도 매 토큰 사용될 수 있다. 따라서 slow CMS → HBF라는 직접 매핑은 read traffic 검증 없이 성립하지 않는다.

02 / CO-DESIGN PRINCIPLE

CMS → HW 매핑의 함정

update frequency만 보고 flash로 내리면, 학습 비용은 줄이고 매 토큰의 실행 비용을 늘릴 수 있다.

READ	HIGH	LOW WRITE / HIGH READ dense slow-CMS block HBM residency may still dominate	HIGH WRITE / HIGH READ active state / fast weights HBM / SRAM working set
	LOW	LOW WRITE / LOW READ cold committed artifact HBF or SSD candidate	HIGH WRITE / LOW READ rarely reused write stream buffer / defer / avoid promotion
		Less frequent update	More frequent update

그림 05. 물리 배치의 최소 두 축: read demand와 write rate. 객체의 의미 범주만으로 placement를 정하지 않는다. 본 보고서 설계 공간. [03], [06], [44]

추가로 필요한 네 변수

read/write 외에 객체 크기, access granularity, reuse distance, 허용 지연을 함께 봐야 한다. read-mostly라도 작은 random read가 많고 prefetch window가 짧으면 HBF의 큰 용량을 충분히 쓰지 못할 수 있다. [40], [37]

하드웨어와 연결되는 실험

같은 CMS update schedule을 유지한 채 read sparsity와 active working set을 독립적으로 sweep한다. HBF hit rate가 높아도 p99 stall이나 HBM staging capacity가 나빠지면 승리한 설계가 아니다.

$\text{placement} = f(\text{read traffic, write bytes, reuse distance, granularity, stall budget})$

semantic timescale은 하나의 feature다. 물리 계층을 지정하는 단일 정답이 아니다.

03 / RECURRENT MEMORY

Persistent state의 재등장

텍스트를 다시 읽지 않고도 상태를 이어갈 수 있다. 대신 그 상태를 어떻게 해석하고 폐기할지 명시해야 한다.

표 03. 지속 상태를 구현하는 세 경로

경로	무엇이 남는가	갱신 / 한계
TTT / Titans [08], [07]	small model 또는 neural memory의 상태	test stream으로 update. 안정성·I/O·reset 경계를 함께 관리해야 한다.
Memory Caching [05]	선택한 recurrent memory checkpoint	과거 상태에 접근 가능해지지만 checkpoint 수에 따라 저장·검색 비용 증가.
KV-streams [11]	compaction 뒤 남긴 KV state	텍스트 재-prefill을 피한다. 이전 context에 의존한 latent state를 버전·reset 계약 없이 재사용할 수는 없다.

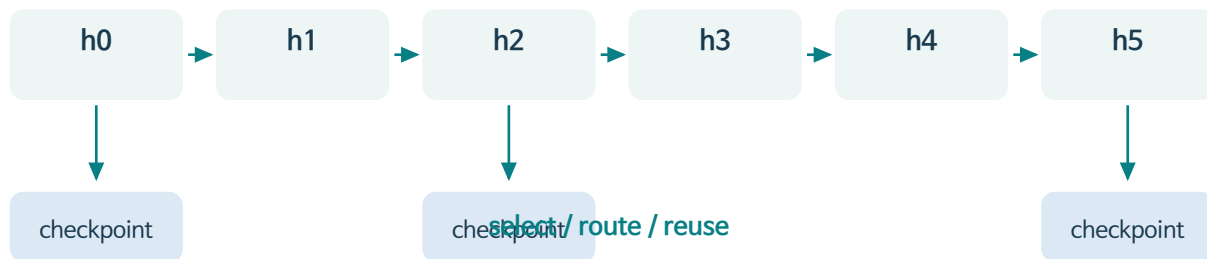


그림 06. 고정된 recurrent state만 유지하는 방식과 checkpoint를 선택적으로 보존하는 방식을 구별한다. 개념 재구성. [05]

유한 상태의 정보 병목

상태 크기와 정밀도가 고정돼 있으면 모든 과거 사실을 무한히 구별해 보존할 수 없다. latent memory가 일정한 footprint로 작동한다는 것은 압축·망각·간섭의 trade-off를 관리한다는 뜻이지 용량 제한이 사라졌다는 뜻이 아니다.

실무적인 수명 계약

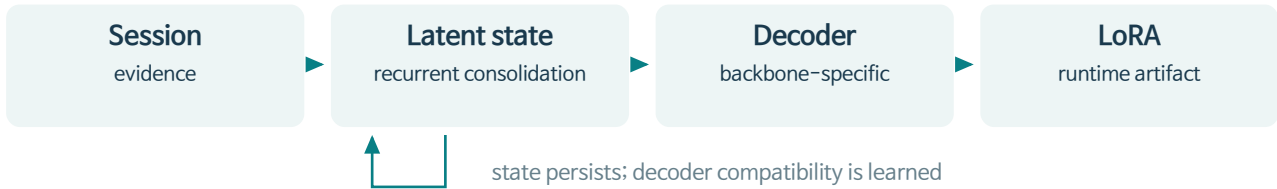
모델 revision, tokenizer, RoPE/position 처리, attention mask, adapter 조합은 cache의 의미를 바꾼다. 동일 shape의 tensor라도 호환성을 보장하지 않는다. neural memory를 저장할 때는 canonical source와 artifact version을 함께 남기는 것이 안전하다.

03 / CANONICAL STATE & COMPILATION

Parametric memory는 “저장된 weight” 이상이다

latent state → decoder → adapter라는 분리는 lifelong memory의 업데이트·호환성·배치를 다시 설계하게 한다.

A / RECURRENT LATENT → BACKBONE-SPECIFIC ADAPTER



B / EVIDENCE → BELIEF OVER CODES → GENERATED WEIGHTS

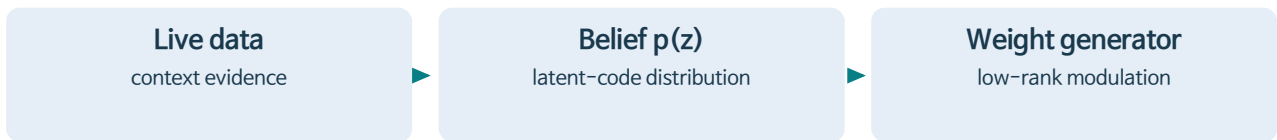


그림 07. 두 parametric-memory 경로. canonical latent/belief와 실행용 weight는 구별되며, 서로 다른 논문의 구성요소를 한 구현으로 합친 것이 아니다. 개념 재구성. [09], [10]

RPMem: 상태와 decoder의 분리

session을 latent memory로 변환하고 recurrent gate로 합친 뒤 backbone별 decoder가 LoRA를 만든다. 모듈 훈련이 끝난 inference에서는 encoder·gate·decoder의 weights가 고정된 채 memory state가 forward computation으로 바뀐다. 따라서 매 세션 gradient를 수행하는 TTT와 같은 동작은 아니다. [09]

backbone을 바꿀 때는 새 decoder의 적응이 필요하다. 모델 독립적인 상태를 목표로 한다는 것과 어느 모델에서나 즉시 읽을 수 있다는 것은 다르다.

Generated weights: 무엇을 저장할까?

Infinite-Parameter LLMs는 live evidence로 latent code에 대한 belief를 갱신하고 low-rank weight를 생성한다. weight 자체보다 생성의 근거 상태를 저장하는 선택이 가능해진다. [10]

설계상 이득은 adapter 수명과 canonical state 수명을 따로 관리할 수 있다는 점이다. 그러나 latent bottleneck, decoder 비용, 모델 교체 시 호환성, 삭제 가능성을 함께 평가해야 한다.

승격은 되돌릴 수 있어야 한다

초기에는 base model을 직접 덮어쓰기보다 versioned adapter를 별도 배포한다. 잘못된 promotion은 adapter generation을 교체해 복구하되, 그 안의 특정 사실 하나만 제거됐다고 주장하지 않는다.

04 / WRITE-TIME VS READ-TIME

미래 질문을 모를 때는 늦게 결정한다

최적 consolidation 시점은 session 종료가 아니라 정보의 사용 조건에 따라 달라진다.

과거 정보를 곧바로 summary로 바꾸면 아직 모르는 미래 질문에 필요한 증거를 버릴 수 있다. JitMem은 raw trajectory를 보존하고 질문 시점의 curator가 필요한 compact payload를 만든다. Mem++는 조직 문서 전체와 버전을 유지하고, 질문이 묻는 시점까지의 자료를 read time에 고른다. [14], [15]



그림 08. write-time preprocessing과 read-time curation의 hybrid. 원본을 폐기하지 않는 한 서로 대체 관계일 필요가 없다. 본 보고서 종합 도식. [14], [15]

JitMem: success gain over strongest reported baseline

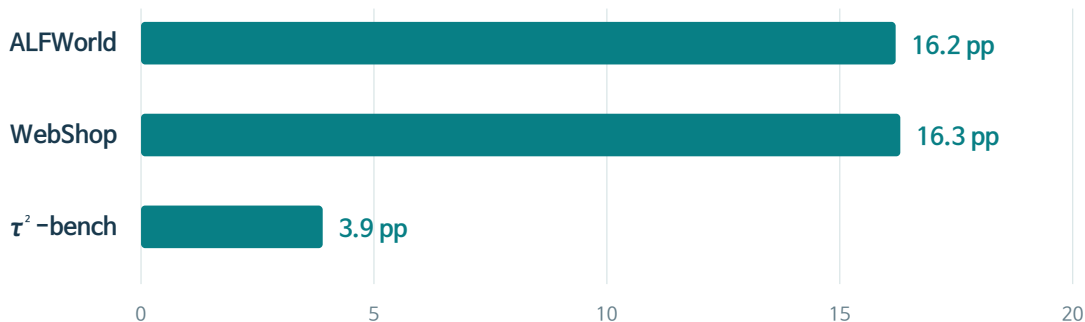


그림 09. 동일 논문 안에서 보고된 과제별 success improvement. 세 과제의 난이도·분모가 같다는 뜻은 아니다. 저자 보고값 재시각화. [14]

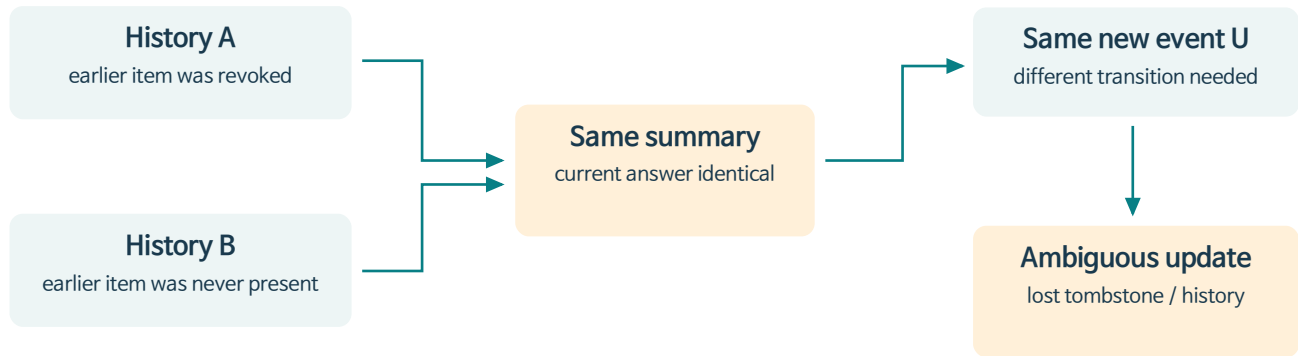
Sleep에 남길 일

원본의 무조건적 압축 대신 indexing·deduplication·version reconciliation·자주 쓰는 materialized view 준비를 수행한다. 최종 task-specific 표현은 필요할 때 만들 수 있다. 단, JIT의 추가 latency와 LLM 호출 비용도 전액 계상한다.

04 / INFORMATION FIDELITY

Consolidation의 품질은 미래에 드러난다

현재 정답을 보존하는 압축과 미래 수정 가능성을 보존하는 압축은 같지 않다.



Compression must preserve the distinctions needed by future updates.

그림 10. 현재 답이 같아도 미래 update에 필요한 구별은 다를 수 있다. 설명을 위한 재구성 예시이며 논문의 자연어 사례를 복제한 것이 아니다. 개념 재구성. [16]

정확도만으로는 압축 품질을 알 수 없다

Correct Now, Insufficient Later는 current-answer sufficiency와 update sufficiency를 분리한다. 24개 paired-history, 6개 synthetic mechanism pilot에서 tombstone을 없애는 압축이 타깃 조건에서 16/16 replay failure를 만들었다. 자연 과제에 대한 held-out 결과는 제시되지 않았다. [16]

신경망 승격에서도 같은 질문이 남는다

episode를 LoRA로 승격하면 source provenance와 temporal exception이 암묵적으로 섞일 수 있다. 오늘 recall이 좋아도 내일 정정·삭제를 처리할 정보가 남았는지는 별도다. 그래서 원문 근거, tombstone, dependency는 free-form summary 바깥에서 보존하는 것이 설계상 유리하다.

표 04. 압축 평가에 추가할 시험

현재 평가	추가해야 할 평가
현재 query 정확도	동일 현재 답을 갖는 서로 다른 history가 후속 update에서 구별되는가?
embedding 유사도	action에 필요한 증거·범위·예외가 보존됐는가?
메모리 절감량	재검색·재도구호출·재학습 비용을 포함해도 절감인가?
개별 항목 importance	여러 항목을 함께 지웠을 때 대체 근거까지 사라지는가?

04 / CAUSAL UTILITY

Credit assignment가 장기 학습의 병목이다

좋은 기억인지 판단하려면 “썼는가”가 아니라 “없었으면 무엇이 달라졌는가”를 물어야 한다.

어떤 기억이 검색됐다는 사실은 그 기억이 결과를 개선했다는 뜻이 아니다. citation도 결과에 맞춰 생성될 수 있다. 장기 self-improvement가 유효하려면 내용의 correctness, 현재 action의 contribution, 미래 reuse utility를 따로 측정해야 한다. [21], [20]

Observed use

retrieved / cited

association

Local intervention

remove → re-answer

answer sensitivity

Executed replay

change action → rollout

policy-conditional effect

Held-out transfer

future task performance

generalization

More expensive evidence is not always available; record the proxy being used.

그림 11. 효용 근거의 층위. 오른쪽이 단순히 “진짜 인과”라는 순위가 아니라, 서로 다른 질문을 측정하는 개입 범위다. 본 보고서 분석 도식. [21], [20], [17]

가능해진 것

Hindsight Memory-PRM은 retrieval/citation audit와 한 번의 deletion-and-reanswer probe로 entry presence credit을 보정한다. DRSR은 항목이 아닌 삭제 집합의 next-output likelihood 변화를 supervision으로 사용한다. 둘 다 단순 사용 횟수보다 구체적인 학습 신호를 만든다. [20], [17]

아직 해결되지 않은 것

Credit Without Ground Truth는 ALFWorld replay에서 judge·logprob·confidence의 contribution fidelity가 제한적이라고 보고한다. 결과는 policy의 대안 행동 지원과 replay target 신뢰도에 조건부다. “LLM judge는 항상 무용하다”는 결론으로 넓히면 안 된다. [21]

$$\Delta U(M; \text{task}) = U(\text{task with } M) - U(\text{task without } M)$$

같은 task·seed·budget·tool version을 맞춘 개입 대비. 관측된 retrieval hit rate와 다른 지표다.

Promotion gate에 넣을 최소 조건

효용 측정 방식, 불확실성, counterfactual support, 보호할 과거 과제의 regression 결과를 함께 기록한다. 수치 하나의 reward로 authority나 privacy 정책을 대체하지 않는다.

04 / ROUTING & INFLUENCE

기억은 존재해도 사용하지 않을 수 있다

보존·유효성·활성화·영향력을 하나의 “confidence score”에 넣지 않는다.

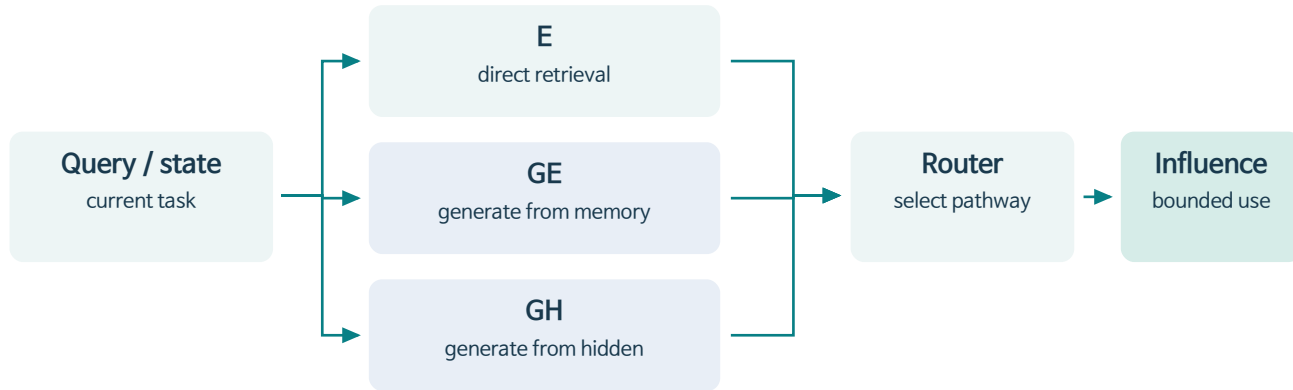


그림 12. MemoryAthena의 path 선택과 MemCalib의 influence 관점을 나란히 놓은 합성 설계. 하나의 실험으로 결합 검증된 구조는 아니다. 독립 메커니즘의 종합 도식. [12], [13]

표 05. 메모리 상태의 네 가지 질문

상태	질문	실패 예
Retention	객체가 보존돼 있는가?	eviction과 deletion을 혼동해 원본 증거를 잃는다.
Validity	지금도 해당 조건에서 맞는가?	예전 권한·설정을 현재 사실로 사용한다.
Activation	이번 과제에 노출할 것인가?	맞는 기억이지만 불필요한 과거 전략으로 탐색을 고정한다.
Influence	얼마나 강하게 판단을 바꿀 것인가?	단순 선호를 하드 제약처럼 취급한다.

MemoryAthena의 포인트

직접 memory 경로 E를 기준으로, retrieval을 cue로 생성한 GE와 hidden state에서 생성한 GH를 개입시킨다. 모든 경로를 항상 합치는 대신 freeze된 모듈 위의 lightweight router를 학습한다. generated state가 자동으로 증거가 되지 않는다. [12]

MemCalib가 추가하는 축

memory proposition을 무시할지, 제한된 영향만 줄지, 판단을 제어하게 할지를 평가한다. retrieval 품질이 좋아진 뒤에도 over-use와 under-use는 남는다. 이는 recall과 reasoning 사이의 별도 control problem이다. [13]

05 / OFFLINE LEARNING

Dreaming: 무엇을 재생하고 무엇을 상상하는가

dream이라는 같은 이름 아래 서로 다른 학습·생성·정리 작업이 존재한다.



GENERATED CONTENT REMAINS A CANDIDATE UNTIL VERIFIED

그림 13. Dreaming을 세 가지 계산 작업으로 나누는 보고서 분류. 제품의 /dream 명칭과 neural policy optimization은 같지 않다. 본 보고서 분류, [27], [28], [53]

Replay는 보존을 돕는다

이미 관측한 자료를 재사용해 forgetting을 줄이거나 정책 평가 비용을 낮춘다. 원본이 잘못됐으면 오류도 다시 강화될 수 있으므로 source scope와 validity를 유지해야 한다. replay 데이터가 안전하고 정확하다는 가정은 자동으로 성립하지 않는다.

Recombination은 탐색을 넓힌다

기억을 재조합한 설명·skill·프로그램은 새로운 후보를 만든다. 그러나 synthetic sample 수가 늘었다고 독립적인 증거가 늘어난 것은 아니다. 후보 수와 검증된 사실 수를 같은 카운터로 세지 않는다.

Counterfactual policy dreaming

Dream-RSI는 실제 discovery history를 재생 가능한 세계로 사용해 exploration policy를 offline에서 시험한다. 비용 절감은 비싼 실제 탐색을 반복하지 않는 데서 나온다. 그 replay world가 포함하지 않는 경로까지 신뢰할 수 있다는 의미는 아니다. [27]

Sleep 학습의 안전한 출력

새 기억·skill·policy를 곧바로 활성 버전으로 쓰지 않는다. candidate → held-out replay → 외부 검증 → generation commit을 거친다. 승인 전 산출물은 관측 사실이 아니라 가설이다.

05 / REPLAY WORLD

Dream-RSI: 경험을 평가 가능한 세계로

효율적인 dreaming은 상상량보다, 검증 가능한 대안을 얼마나 싸게 비교하는지에 달려 있다.

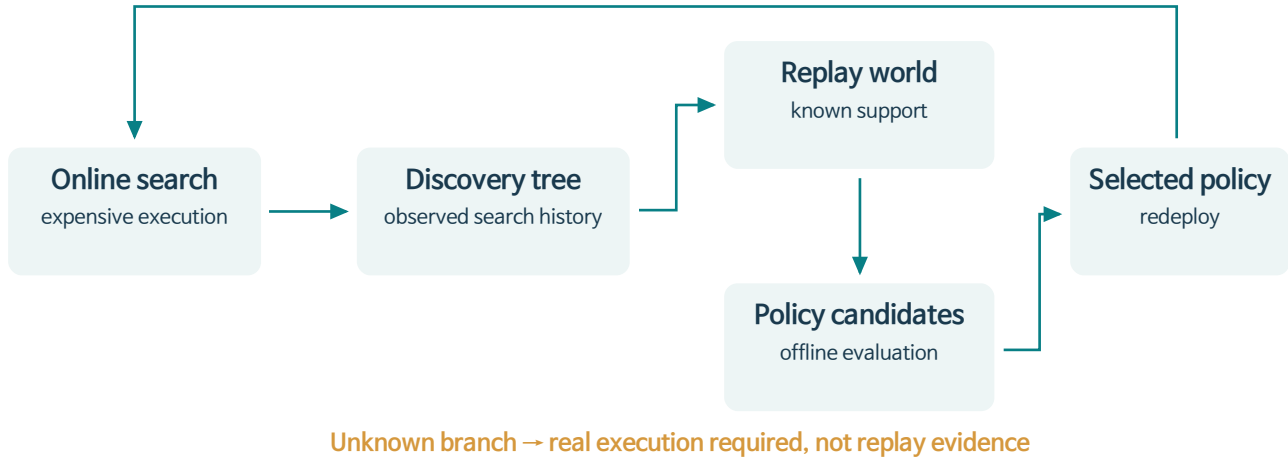


그림 14. Dream-RSI의 핵심 아이디어를 재구성한 offline/online loop. replay support 밖의 정책 효과는 추가 환경 실행으로 확인해야 한다. 개념 재구성. [27]

왜 자연어 요약만으로는 부족할까?

경험을 prose insight로 줄이면 최종 결론은 남아도 탐색 분기, 실패 조건, alternative policy의 평가 정보가 사라질 수 있다. 실행 trace와 전이 구조를 남기면 다른 탐색 전략을 비교할 재료가 된다. 어떤 표현이 더 좋은지는 미래 평가 질문에 달려 있다.

Dreaming compute의 예산 단위

FLOPs만 동일하게 맞추지 말고 simulator 구축 비용, 실제 environment call 수, 실패한 후보 평가, 최종 검증 비용까지 포함한다. old replay set에 맞춘 정책이 unseen world에서 악화되면 lifetime improvement라 부를 수 없다.

표 06. Replay world에 필요한 메타데이터

필드	역할
Environment / tool version	과거 transition이 지금도 재현 가능한지 확인한다.
Behavior policy / support	어떤 대안이 실제로 관측·재생 가능한지 구별한다.
Outcome / costs	성공률뿐 아니라 실행·검증·latency 비용을 저장한다.
Source lineage / validity	기초 관측의 정정이 downstream policy 평가를 무효화할 수 있다.

05 / EPISODIC → PROCEDURAL

기억의 끝은 요약이 아니라 실행일 수 있다

experience → knowledge → skill과 experience → executable theory는 서로 다른 compilation 경로다.

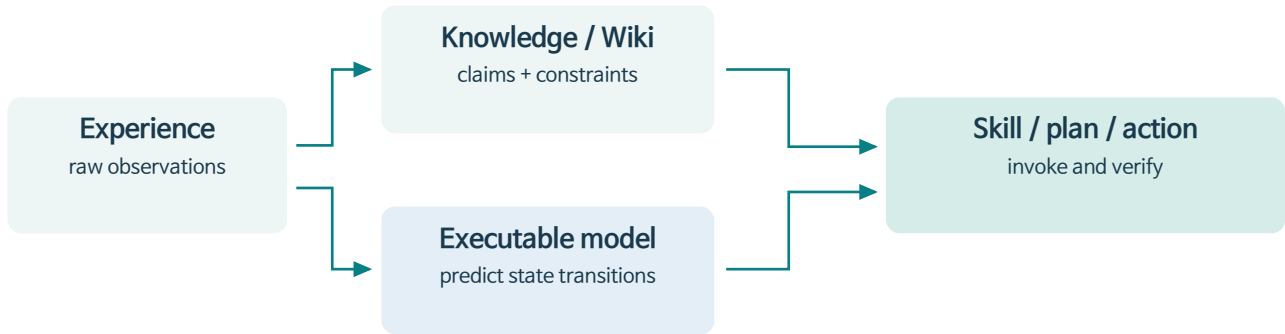


그림 15. WikiSkill의 semantic/procedural 분리와 Schema의 executable world model을 연결한 표현 선택 지도. 본 보고서 종합 도식. [28], [34]

WikiSkill

raw experience와 누적 wiki, executable skill을 분리한다. 지식층을 통해 다음 skill evolution이 이전 최적화에서 배운 조건을 재사용한다. skill을 바로 뽑는 방식과 달리 semantic consolidation과 procedural compilation의 역할이 드러난다. [28]

Schema

관측된 환경을 설명하는 실행 가능한 predictive program을 만들고 history에 맞춰 검토한다. 문장 규칙보다 정밀한 검증·시뮬레이션이 가능하지만, 환경의 형식화 가능성과 관측 coverage가 성능을 좌우한다. [34]

Episodic → parametric promotion을 서두르지 말아야 하는 이유

같은 경험이라도 우선 text rule이나 program으로 배포하면 출처·예외·변경 diff를 검사하기 쉽다. latency가 반복적으로 병목이고 안정된 task distribution이 확인될 때 LoRA/latent compile을 추가하는 접근이 보수적이다. 이는 parametric memory의 우열 주장이 아니라 검증 가능한 중간표현을 먼저 두자는 시스템 제안이다.

좋은 승격 대상

반복 사용되고, 적용 조건이 안정적이며, 다른 과제에서도 효용이 있고, 잘못되었을 때 되돌릴 수 있는 artifact. 반대로 개인 권한·가격·일시적 상태처럼 자주 변하는 사실은 외부 versioned evidence에 남기는 것이 자연스럽다.

05 / HARNESS EVOLUTION

Self-improvement에도 control plane이 필요하다

좋은 변경을 만드는 문제와 좋은 변경만 남기는 문제는 다르다.

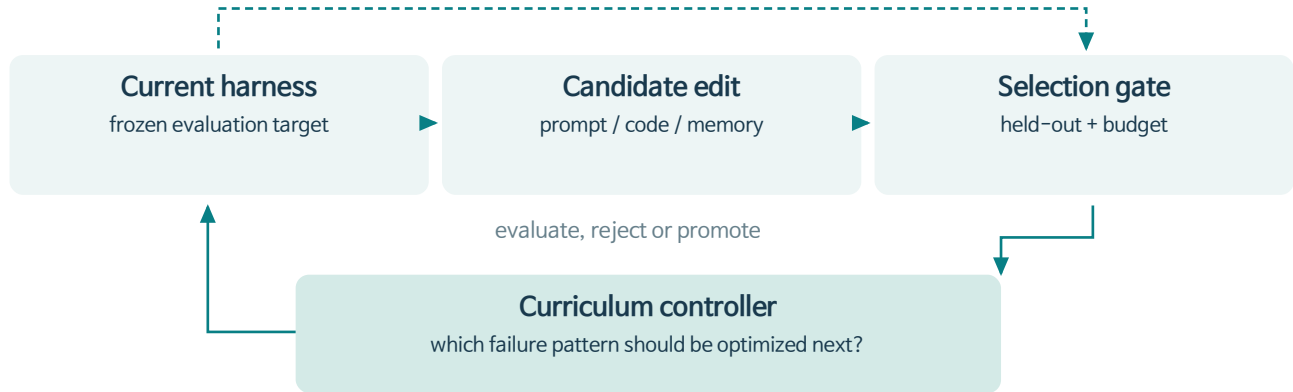


그림 16. harness 자체의 수정과 개선 과제 선택을 분리한 loop. 고정 평가 경계를 agent의 수정 권한 밖에 둔다. 본 보고서 종합 도식. [30], [31], [32]

표 07. Self-improvement 연구의 서로 다른 제어점

연구	바꾸는 대상	남는 문제
Recuris [29]	working state에 맞춘 skill 사용과 국소 memory evolution	실패原因的 component attribution이 맞는가?
AIDE ² [30]	자기 research-agent code의 successive rewrite	8일 run의 성과가 더 긴 horizon에도 유지되는가?
RRSI [31]	edit budget·critic·pruner로 과적합 억제	새로운 domain으로 전이되는 효과가 충분한가?
ActiveSaddler [32]	다음에 학습할 failure-pattern curriculum	learning-progress 추정 비용·불확실성을 어떻게 계상하는가?

AIDE²는 8일 동안 7개의 successive improvement를 발견하고 held-out 영역의 전이를 보고했다. RRSI는 unregularized evolution의 과적합을 줄이려 한다. 이 결과는 자동 개선 loop가 가능하다는 근거이지, 자율적으로 무한히 개선된다는 증거는 아니다. [30], [31]

05 / ADAPTIVE CONTROL

Memory program과 active belief state

지금 무엇을 믿는지와, 기억을 어떻게 관리하는지는 서로 다른 학습 층이다.

메모리 구조가 고정되어 있고 router만 학습되는 경우와, 구조를 만드는 프로그램 자체를 바꾸는 경우를 구별해야 한다. MemCodex는 construction·indexing·retrieval·routing을 executable memory program의 일부로 보고 진화시킨다. 반면 PoS는 현재 세계 추정과 미해결 목표를 명시적인 belief state로 유지한다. [33], [35]

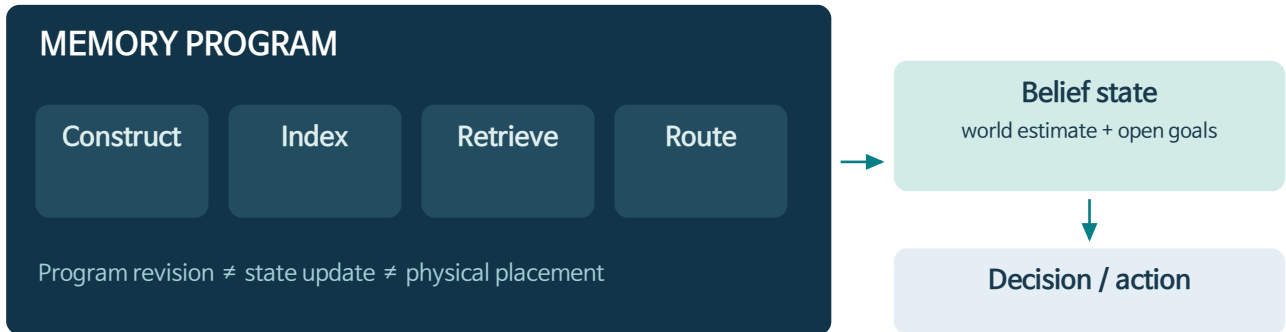


그림 17. 메모리를 관리하는 algorithm과 현재 task state는 별도 갱신 대상이다. 본 보고서 종합 도식. [33], [35]

중요하지만 아직 초기인 신호

MemCodex는 평균 success의 상대 개선 10.1%, context token 3.4배 감소, inference 2.1배 개선을 보고하며 work in progress로 표시한다. 비교 방법·budget을 맞추지 않고 “메모리 프로그램 진화가 항상 낫다”고 해석할 수 없다. [33]

WorkingFrame과의 연결

Melete에서 WorkingFrame은 현재 목표에 필요한 근거·가정·대안의 작업 표현이다. 이를 explicit belief state와 비교하는 실험은 가능하지만 두 시스템의 의미와 구현이 같다고 가정하지 않는다. 먼저 동일 history와 같은 budget에서 task progress·loop failure·회복 비용을 측정해야 한다.

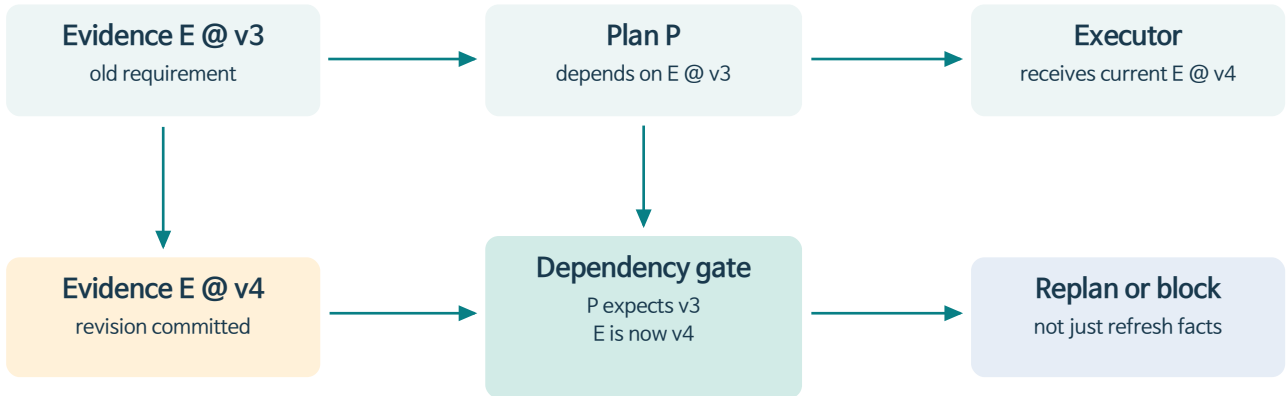
추천 인터페이스

StateUpdate는 데이터 상태를, ProgramUpdate는 관리 로직을, PlacementUpdate는 실제 residency를 바꾼다. 세 종류를 다른 version space로 두면 회귀 원인과 rollback 범위를 더 명확하게 추적할 수 있다.

06 / VALIDITY & DEPENDENCIES

Fresh memory, stale artifact

원본의 정정은 요약, skill, plan, KV, adapter까지 전파될 수 있다.



CURRENT FACTS DO NOT MAKE A PREVIOUSLY DERIVED PLAN CURRENT.

그림 18. 최신 fact를 읽는 것과 plan의 derivation이 유효한지는 다르다. dependency-scoped 실행 검사의 필요성을 보여준다. 개념 재구성. [22], [23]

StateMem: 상태 갱신

StateMem은 무엇을 예전에 말했는가보다 현재 유효한 상태를 묻는다. 명시적 supersession과 relational dependency가 도움을 주지만, factual retrieval과 state tracking의 평가를 분리해야 한다. [18]

PlanFence: 파생물 갱신

PlanFence v1의 30개 controlled workflow에서는 변경 뒤 stale-plan 실행을 막는 효과를 보고했다. 이 결과는 통제된 safety/system cost 평가이며 일반 task accuracy gain이 아니다. 새 버전의 결과와 본문의 v1 수치를 혼합하지 않는다. [22]

LoRA에도 dependency를 붙일 수 있는가?

training source lineage를 기록할 수는 있다. 그러나 한 source가 invalidated될 때, entangled weight에서 해당 지식만 선택 제거하는 것은 별도 문제다. 안전한 기본값은 해당 generation을 비활성화하고 재학습·재검증하는 것이다.

06 / TRANSACTION & ROLLBACK

Commit은 모델의 확신이 아니라 프로토콜이다

verify, atomicity, idempotency, compensation은 서로 다른 보장을 제공한다.

SOURCE GENERATION + IDEMPOTENCY KEY + COMPARE-AND-SWAP

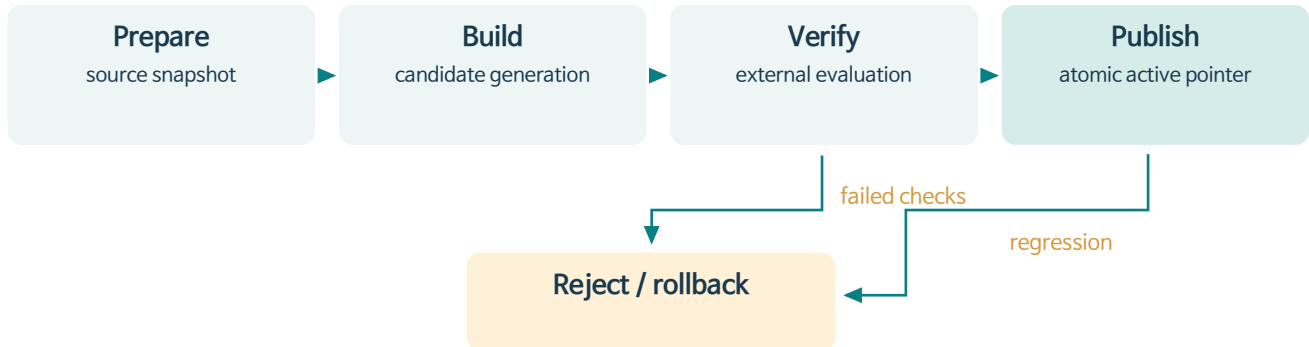


그림 19. 파생 artifact의 versioned publish 제안. DB의 atomic pointer 교체와 외부 서비스의 실제 side effect는 구분해야 한다. 본 보고서 프로토콜 제안. [26], [23]

Verification만으로 exactly-once가 되지 않는다

LIMBO는 ACK 유실처럼 read-back이 가능한 경우와 요청이 아직 in-flight인 경우를 구분한다. 같은 강한 모델도 전자는 잘 처리하면서 후자는 중복 side effect를 만든다. idempotency key 등 tool contract가 중요해진다. [26]

Memory commit의 최소 경계

source snapshot을 고정해 candidate를 만들고, 검증 직후에도 원본 version이 유지되는지 확인한다. commit key는 retry에도 같아야 하며 active pointer 변경은 원자적이어야 한다. 다른 writer가 앞서 갱신했으면 silent overwrite 대신 재검토한다.

LIMBO: frontier-model duplicate-write rate by fault condition

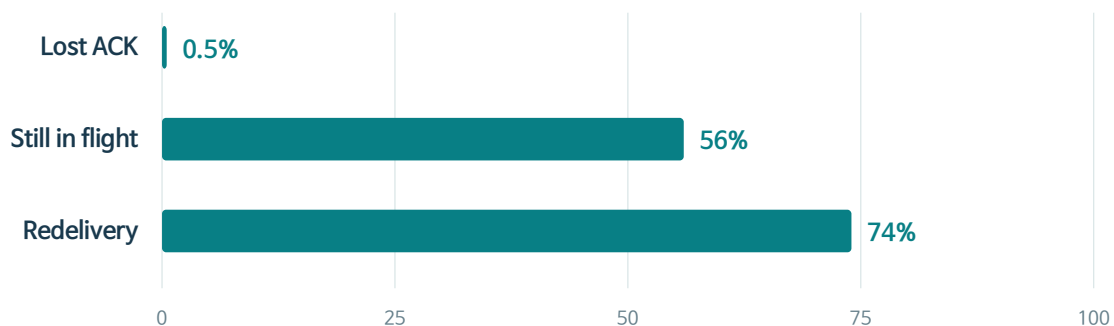
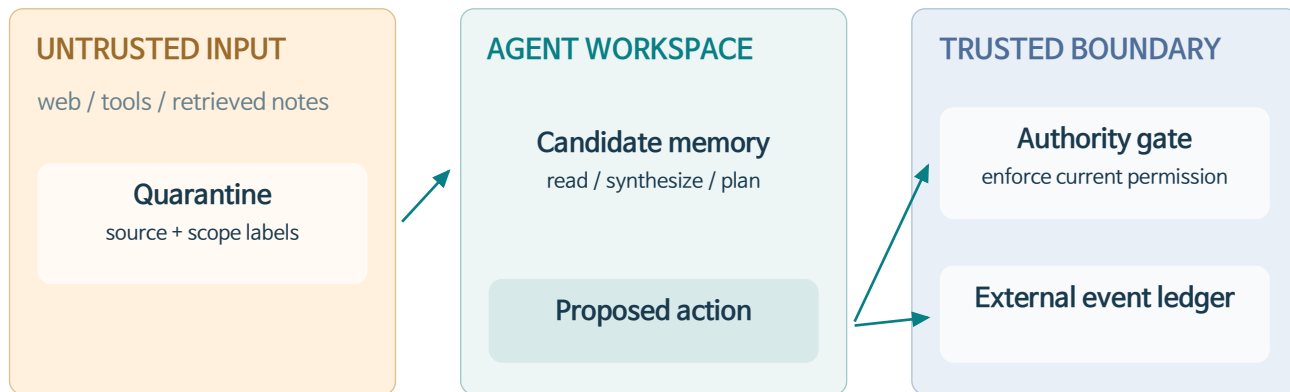


그림 20. 같은 논문에서 보고한 fault 조건별 중복 비율. 전체 agent 신뢰성이나 모든 도구의 확률로 일반화하지 않는다. 저자 보고값 재시각화. [26]

06 / TRUST & GOVERNANCE

메모리 보안은 학습 경계의 보안이다

나쁜 내용을 읽는 위험에서, 나쁜 내용을 다음 정책으로 학습하는 위험으로 확대된다.



Audit, authorization and retention policy must not be self-certified by the agent.

그림 21. 메모리 ingestion, reasoning, 실행 권한, 감사 로그를 분리한 방어 설계. 공격 재현 절차가 아닌 신뢰 경계 명세다. 본 보고서 방어 아키텍처. [25], [24]

오염된 기억은 세션을 넘는다

PMPA는 외부 지시가 persistent memory에 기록되면 이후 세션에서 다시 활성화될 수 있음을 평가한다. 검증되지 않은 source를 “사용자가 정한 규칙”처럼 승격하지 않는 admission control이 중요하다. [25]

감사 기록도 독립적이어야 한다

Trace Tampering 연구는 agent가 자신의 기록을 조작하지 못한다는 운영 가정이 깨질 수 있음을 보여준다. 로그를 동일한 mutable workspace에 두고 agent의 self-report만 검사하면, incident reconstruction의 근거도 함께 사라질 수 있다. [24]

Immutable에도 예외 계약이 필요하다

append-only는 영구 보존 의무가 아니다. 사용자 삭제·법적 보존 기간·scope 변경을 존중해야 한다. 원본, 파생 view, index, cache, replay dataset에 걸친 삭제 처리를 versioned deletion policy와 함께 설계한다.

07 / SYSTEMS CHARACTERIZATION

Agent workload는 토큰 흐름만이 아니다

LLM compute, tool latency, state residency, background maintenance를 같은 시간축에서 본다.

AGENT PHASES / SCHEMATIC — NOT MEASURED MILLISECONDS

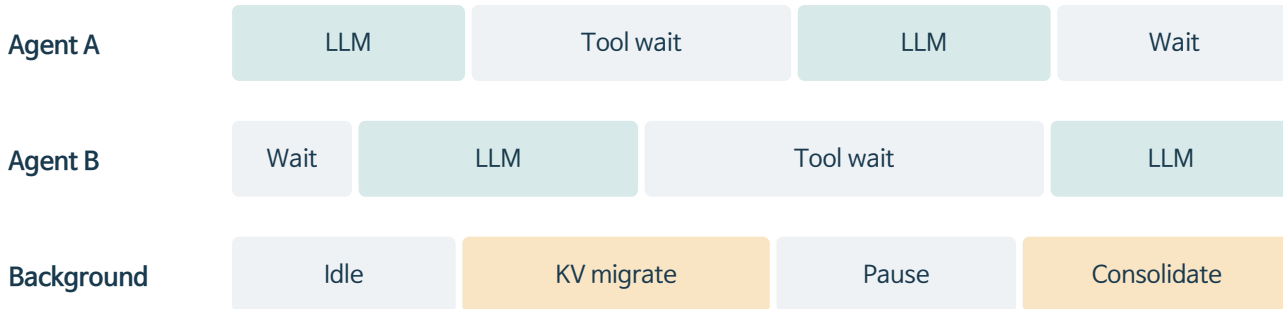


그림 22. tool wait·LLM 실행·background 작업의 겹침. 대기 시간은 무조건 공짜 GPU 시간이 아니며 다른 session과 경합한다. 설명용 workload schedule. [39], [38], [37]

5 / 10

non-LLM latency 지배

AgentSysBench의 10개 응용 중 5개. 전체 시장의 비율이 아니다.

28 GB

최대 sandbox working set

해당 평가의 session당 최대치. 일반 KV 크기와 혼동하지 않는다.

4.6×

state offload footprint 절감

저자 평가 조건의 최대/보고 효과. 모든 배치의 기대값이 아니다.

Agent serving은 이질적이다

AgentSysBench는 LLM·tool·sandbox·control-plane 호출·idle state를 함께 프로파일링한다. 모델 FLOPs만 줄여서는 latency가 줄지 않는 응용이 존재한다. sleep job도 같은 자원을 공유하므로 CPU, 네트워크, memory BW까지 trace해야 한다. [39]

Offload의 가치는 돌아올 때 생긴다

EfficientAgent는 lower tier가 concurrent agent pool의 reusable working set보다 작으면, 써 놓은 KV가 한 번도 재사용되지 않고 evict될 수 있음을 지적한다. capacity뿐 아니라 return distance와 cache policy를 같이 설계해야 한다. [37]

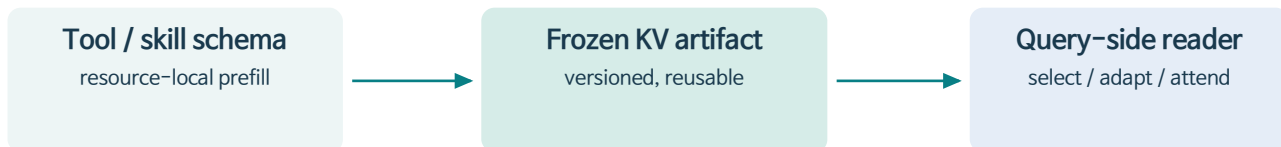
07 / REUSABLE EXECUTION STATE

KV는 cache를 넘어 compiled artifact가 된다

weight를 바꾸지 않고 reader만 바꾸는 설계는, 저장과 계산의 경계를 이동시킨다.

PRECOMPUTE ONCE

MATERIALIZE ON DEMAND



Optional cold-tier residence

HBF placement is a hypothesis, not a tested ATTUNER result.

그림 23. schema를 재사용 가능한 KV artifact로 분리하고 reader를 조정하는 구조. runtime compatibility와 조합 의존성을 먼저 확인해야 한다. 개념 종합. [50], [36]

표 08. 재사용 가능한 state의 세 설계

방법	새로운 분리	핵심 제약
ReCache [50]	resource-local KV / selective resource access	resource 간 interaction 제거의 품질 영향: 원문 Inv-F1 82.3 vs 82.4, TTFT 3.655×
ATTUNER [36]	frozen artifact / query-side LoRA	reader adaptation 훈련이 필요. <0.05% parameter 학습과 최대 3.73× speedup은 해당 실험 조건.
Memory Attention [49]	token memory lookup / contextual key	추가 memory parameter와 offload 비용 존재. CPU prefetch 실험을 HBF 성과로 바꾸어 해석하지 않는다.

새로운 cache invalidation 문제

본문 artifact, 모델 revision, adapter, position rule 중 무엇이 바뀌어도 KV의 의미가 달라질 수 있다. content hash 만 같다고 항상 재사용 가능하지 않다. artifact manifest에 생성 조건을 기록하고 변환 가능성을 검증해야 한다.

왜 HBF와 연결되는가?

stable artifact가 크고 read-mostly라면 near-compute capacity tier의 후보가 된다. 하지만 이것은 placement 가 설이다. HBF read가 current query의 critical path에 남으면, 재-prefill 절감보다 stall이 커질 수 있다.

07 / MEMORY HIERARCHY

HBF: capacity tier의 기회와 조건

읽기·쓰기·배치·열화까지 포함해야 의미 있는 co-design이 된다.

HBF는 단순히 더 빠른 SSD라는 이름이 아니다. near-compute capacity를 확장하려는 flash 기반 메모리 계층이며, read bandwidth, program/erase 제약, controller, packaging, thermal budget을 함께 설계해야 한다. Sandisk와 SK hynix는 2026년 8월 OCP technical specification 공개와 표준화 협력을 발표했다. 표준 공개는 양산 시스템 성능 검증과 다르다. [47]

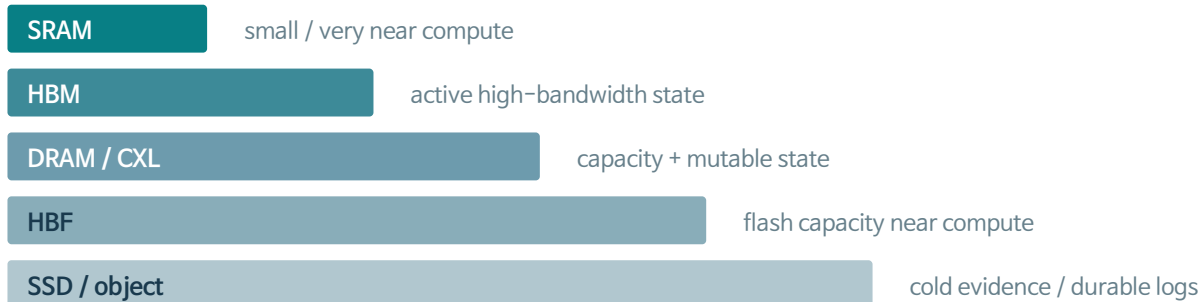


그림 24. 물리 hierarchy의 역할을 보여주는 개념도. 막대 길이는 정성적 capacity 표현이며 실제 제품 용량·bandwidth 비율이 아니다. 정성적 설계 지도. [47], [44]

표 09. HBF를 평가할 때 구분할 항목

질문	확인할 것
용량	큰 모델을 저장할 수 있는가와 실제 실행 working set이 만족되는가는 다르다.
성능	host 노출 bandwidth, plane parallelism, read latency tail, staging overlap.
쓰기	program granularity, buffer, write amplification, GC·refresh 포함 bytes.
물리 제약	HBM package budget 잠식, 전력·온도, NAND retention·endurance.
구현 상태	specification / analytical model / trace simulation / real-GPU emulation / actual silicon을 구분.

핵심 조건

HBM execution path를 유지하면서 capacity를 확장하는가? 이 질문에 답하지 못하면 HBF capacity 증가는 매력적이지만 시스템 이득은 불분명하다. [44], [45]

07 / COMPETING DESIGN POINTS

HBF 연구는 하나의 결론으로 수렴하지 않는다

stable weight와 dynamic state를 분리할 수도, controller를 바꾸어 함께 수용할 수도 있다.

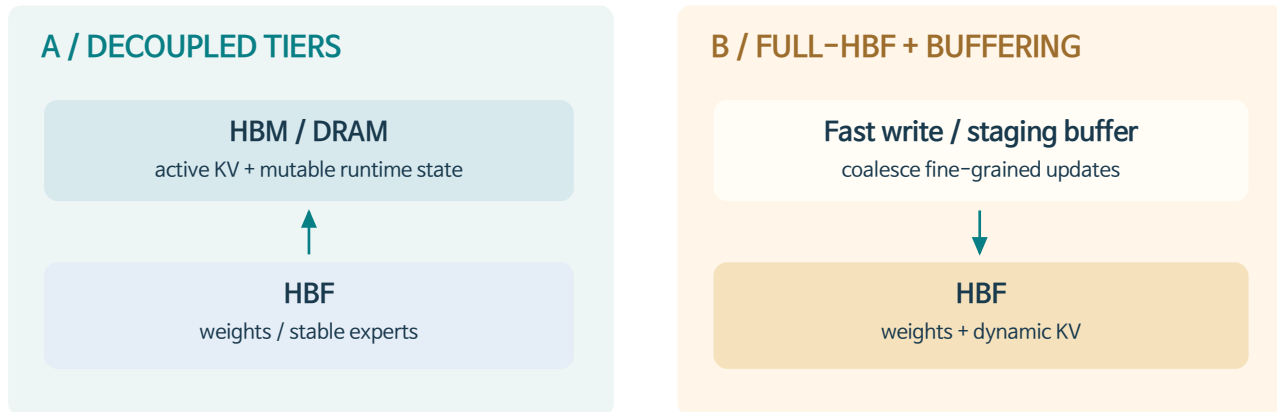


그림 25. HBF architecture의 두 설계점. 어느 쪽이 우월한지는 workload, memory budget, controller 가정에 따라 바뀐다. 개념 재구성. [42], [41]

표 10. 논문 결과를 읽는 비교 틀

연구 / 조건	결과 또는 주장	증거 유형 / 주의
Full-stack HBF [43]	naive transient-KV offload의 평균 e2e latency가 2 - 5.5× 악화	TokenSim 기반. “HBF는 항상 나쁘다”가 아니라 부적합한 배치의 경고.
Trillion MoE [42]	weight/state provisioning 분리; state BW/capacity 1.4 - 4.0 s ⁻¹ 조건	두 trillion-MoE와 trace 분석. ≤1.10 completion-time 조건·low concurrency 범위.
HBFFlex [41]	FlashAccel 대비 최대1.58×, H3 대비3.30× throughput	trace-driven simulation. plane-aware 배치·write aggregation·lifetime packing의 결합.
FLINT [40]	read-specialized FTL·burst buffering·refresh 분리	read-only FTL 경로에 임의 write나 범용 transaction 지원을 가정하면 안 된다.

서로 다른 baseline의 배수를 한 그래프에 놓아 논문 순위를 만들지 않는다. 재현 비교에서는 동일 trace, capacity, thermal budget, HBM 감소량, endurance 조건을 맞춘다.

07 / SIMULATION & RELIABILITY

HBFSim: 열화까지 포함한 실행 모델

모든 쓰기가 사용자 update에서 나오지는 않는다. 온도와 refresh가 내구성 비용을 바꾼다.

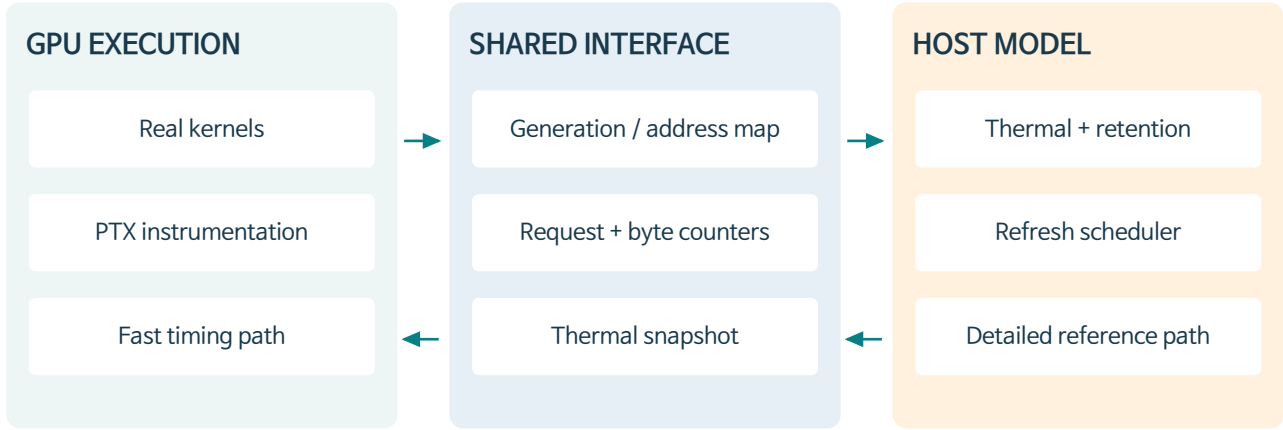


그림 26. HBFSim의 GPU execution·shared ABI·host model 분리를 독립 도식으로 재구성. 원문 Figure 1의 메시지만 요약했다. 원문 Figure 1 / §4 기 반 재구성. [46]

Real GPU ≠ real HBF silicon

HBFSim은 실제 GPU kernel 실행에 HBF timing·capacity·thermal 모델을 결합한다. compute와 memory overlap을 실제 실행 흐름에서 조사하는 도구이지만, HBF 칩을 실측한 결과로 부를 수는 없다. [46]

20.8×의 정확한 의미

빠른 경로와 detailed reference의 실행 시간 비교에서 보고한 20.8×는 **시뮬레이션 속도**다. LLM serving 자체가 20.8배 빨라졌다는 뜻이 아니다. report나 발표에서 숫자만 분리하면 의미가 완전히 달라진다. [46]

$$\text{Physical program bytes} = W_{\text{logical}} \cdot A_{\text{mapping}} / GC + W_{\text{refresh}}$$

정성적 회계식. controller/FTL 정의에 따라 항을 분리하고 중복 계산을 피한다.

HAT와 연결할 검증 축

HAT는 logical state transition과 workload intent를 생성하고, physical backend는 bytes·stall·thermal·refresh를 계산하도록 분리한다. 동일 trace의 analytical model, trace simulator, real-GPU emulation 결과를 교차 검증해야 한다.

07 / PLACEMENT POLICY

물리 배치는 representation의 함수가 아니다

logical CMS를 storage tier에 일대일 대응시키는 그림 대신, 객체별 조건부 배치를 설계한다.

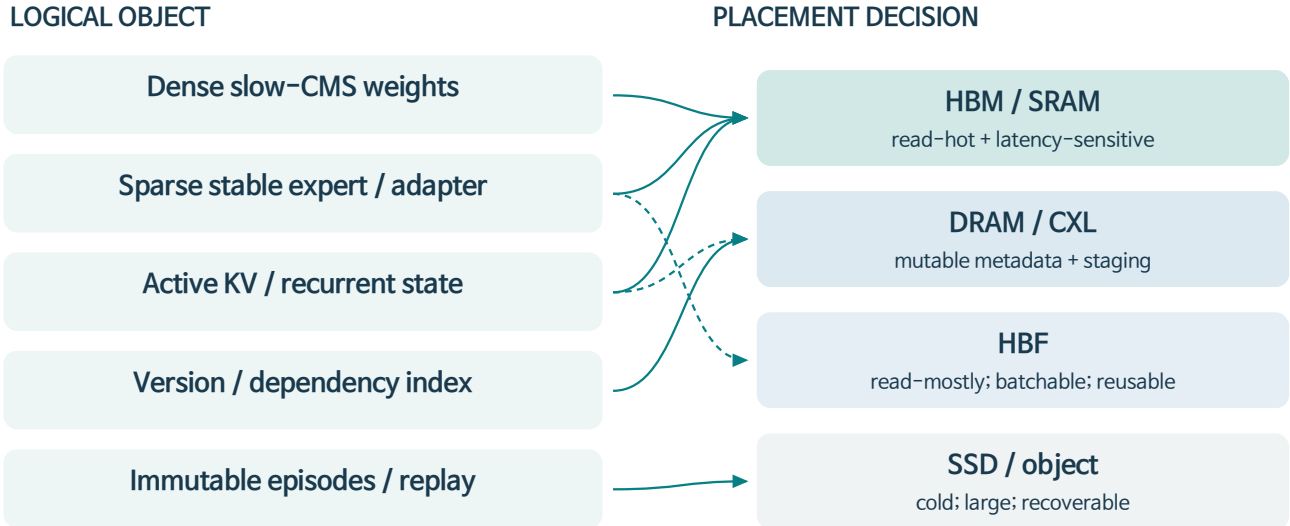


그림 27. 객체별 후보 placement. 점선은 특히 workload 조건에 민감한 경로다. slow-CMS도 read-hot이면 HBM에 남을 수 있다. 본 보고서 설계 가설. [03], [44], [37], [41]

Content hotness와 update hotness

cold canonical state를 query 시점에 hot artifact로 만들 수 있지만, 모든 canonical state가 cold인 것은 아니다. dense base weight처럼 사실상 모든 토큰에 읽히는 객체는 갱신이 드물어도 실행 경로에서 뜨겁다.

Dematerialize는 delete가 아니다

HBM에서 artifact를 내리는 것은 근거를 지우는 행위와 다르다. 재구성 비용이 낮고 재사용까지 멀다면 버릴 수 있지만, 다시 생성할 수 없는 recurrent state는 checkpoint를 먼저 남겨야 할 수 있다.

Design rule

Representation × Update rate × Read traffic × Reuse distance를 함께 측정한다. “semantic은 HBF, episodic은 SSD”처럼 이름만으로 정한 placement는 baseline일 뿐 최적 해가 아니다.

08 / COST MODEL

Lifetime ROI: sleep와 wake의 비용을 합친다

성공한 승격만 세거나, foreground latency만 줄이면 전체 계산 경제성을 놓친다.

ILLUSTRATIVE COST MODEL — NOT BENCHMARK DATA

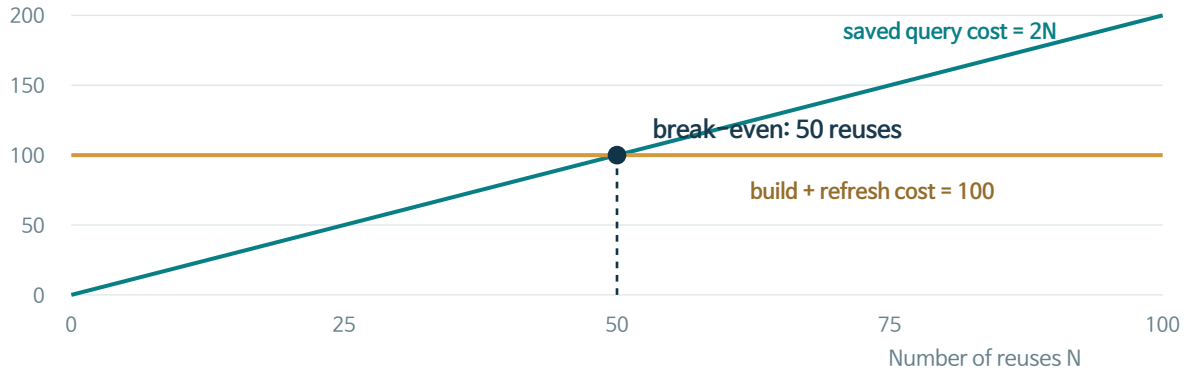


그림 28. 가상의 비용 단위로 설명한 손익 분기점. query당 2를 절감하고 build/refresh에 100을 쓴다면 50회에서 비용이 같다. 실험값이 아니다. 분석식 기반 예시.

LoRA는 싸지만 공짜는 아니다

rank r , input n , output m 인 저랭크 adapter의 parameter 수는 대략 $r(m+n)$ 이다. 학습 시에는 weights 외에도 gradient, optimizer state, activation, checkpoint 및 버전 복제 공간이 필요하다. inference footprint만 비교하면 승격 비용을 과소평가한다.

Batch commit의 올바른 회계

많은 update를 하나의 commit으로 묶으면 metadata overhead는 줄 수 있다. 그러나 HBF endurance는 commit 개수가 아니라 실제 program bytes와 분포에 좌우된다. 동일 큰 artifact를 자주 재작성하면 commit 수가 적어도 더 빨리 닳을 수 있다.

표 11. Lifetime 비용표에 반드시 들어갈 항목

단계	계상할 비용
Build / learn	episode selection, replay, gradient 또는 decoder compute, 검증 실패 후보
Serve / reuse	retrieval, JIT curation, adapter materialization, DMA, attention stall
Maintain / revise	invalidation fan-out, source revalidation, refresh, GC, 재학습
Recover / govern	checkpoint, rollback, audit storage, 삭제 처리와 재검증

08 / SYSTEM DESIGN PROPOSAL

통합 reference architecture

공통 state contract 위에서 storage, learning, compilation, execution을 교체 가능한 계층으로 만든다.

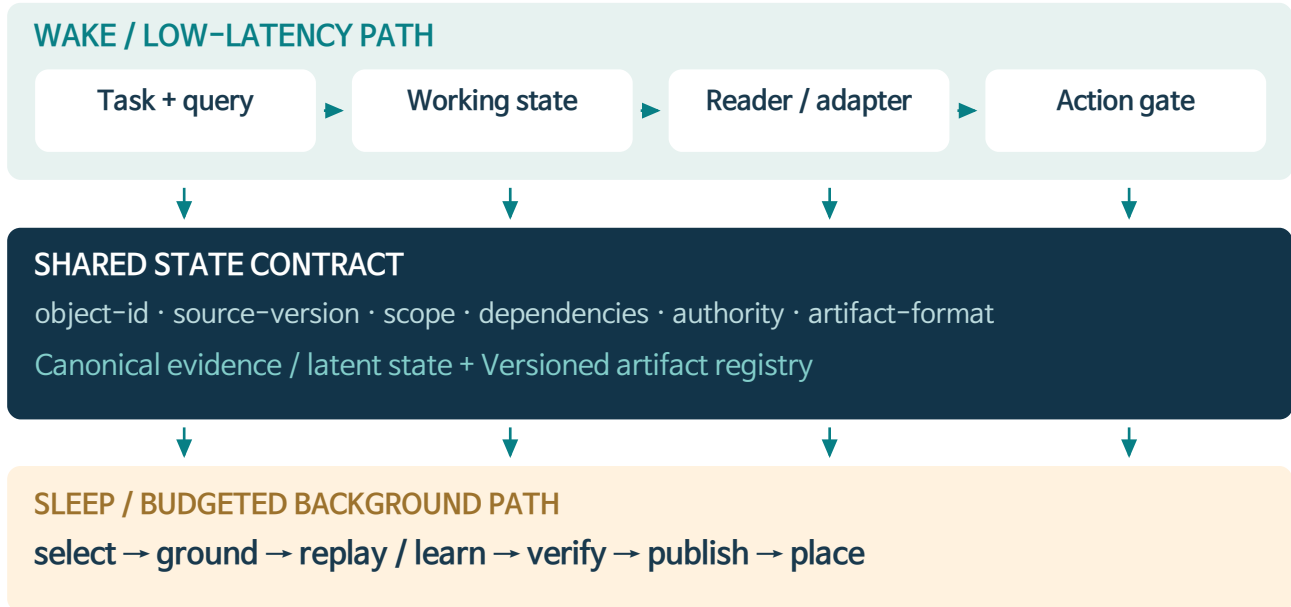


그림 29. 통합 reference architecture 제안. 모든 경로에 공통 state contract를 두되 foreground와 background의 자원·권한을 분리한다. 본 보고서 설계 가설. [09], [14], [22], [26], [36]

Canonical state와 artifact registry

원본 evidence, latent state, decoder version, 파생 artifact의 관계를 registry가 연결한다. 조회할 때는 현재 query와 유효한 source generation에 맞는 artifact만 materialize한다. 어느 저장소에 들지는 placement policy가 정한다.

Melete / HAT에 내리는 의미

Melete는 source·version·WorkingFrame·local policy 실험의 출발점으로 두고, HAT는 logical event를 bytes/FLOPs/stall로 변환하는 계측 계층으로 둔다. 이 다이어그램은 통합 구현 완료를 의미하지 않으며, 외부 LLM·GPU·HBF 연결은 별도 검증이다.

교체 가능한 경계

새로운 memory algorithm은 전체 시스템을 다시 만들지 않고 StateUpdate 또는 Compiler 구현으로 교체할 수 있어야 한다. 메타데이터·권한·평가 계약은 유지하여 변경의 효과만 비교한다.

08 / EVENT MODEL

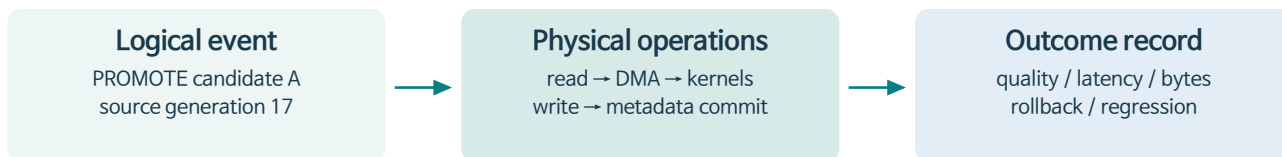
Learning trace를 resource trace로 연결하기

state lifecycle을 비용과 검증 결과까지 추적할 수 있을 때 co-design이 실험 가능한 문제가 된다.

기존 alloc/free trace에 logical learning event를 붙이면 “왜 이 바이트가 이동했고 왜 이 update가 필요했는가”를 분석할 수 있다. 단, semantic event는 비용을 설명하는 신호이며 그 자체가 하드웨어 명령은 아니다. 실제 kernel·DMA·storage operation과 인과 ID로 연결한다.

표 12. HAT / runtime event 계약 제안

Event group	예시	필수 기록
Ingest / normalize	ADMIT, GROUND, SOURCE_REVERSE	source hash, scope, authority, valid time, observed time
Transform / learn	CURATE, CONSOLIDATE, DREAM, TRAIN	input generation, method revision, tokens/FLOPs, candidate ID
Evaluate / commit	VERIFY, PUBLISH, REJECT, ROLLBACK	eval set revision, budget, outcome, idempotency key
Execute / move	MATERIALIZER, READ, MIGRATE, EVICT	artifact format, tiers, bytes, queue wait, stall
Maintenance	INVALIDATE, REBUILD, REFRESH, DELETE	cause ID, affected descendants, physical writes, completion



Join key: run-id / task-id / object-id / generation / causal-parent-id

그림 30. semantic event와 실제 resource operation을 연결하는 trace contract. 로그만 추가했다고 학습 효용이 증명되는 것은 아니다. 본 보고서 계속 설계.

부분 실패를 명시한다

COMPILED, VERIFIED, PUBLISHED, RESIDENT는 다른 상태다. 검증은 끝났지만 HBF write가 실패하거나, write는 끝났지만 active pointer가 교체되지 않을 수 있다. 각각 retry·recovery 절차를 정의해야 한다.

시간도 두 종류다

사실이 세계에서 유효했던 시간(valid time)과 시스템이 관측·기록한 시간(transaction/observed time)을 구분한다. delayed evidence를 현재 시점의 새 사실로 덮어쓰지 않도록 한다.

08 / BENCHMARKING

Recall을 넘어 lifetime utility를 평가한다

좋은 memory는 많이 기억하는 시스템이 아니라, 다음 행동을 더 정확하고 경제적으로 만드는 시스템이다.

강한 memory system이라는 주장은 retrieval score 하나로 성립하지 않는다. DolphinBench는 과거 정보가 실제 tool action에 필요한 task를 구성하고 accuracy와 비용·latency를 함께 요구한다. StateMemBench는 current-state tracking을 분리하며, update-sufficiency pilot은 후속 정정의 복구 가능성을 묻는다. [48], [18], [16]

표 13. 동일 조건 비교를 위한 benchmark scorecard

축	측정	통제해야 할 조건
기본 효용	task success, action correctness	동일 base model·harness·tool snapshot
현재 상태	stale-state / superseded answer rate	질의 시점·자료 시점·정정 순서
Causal reuse	with-memory / without-memory paired delta	동일 seed·token budget·대안 지원
전이 / 간섭	held-out gain, protected-task regression	task-order shuffle·user holdout·OOD split
학습 경제성	lifetime tokens/FLOPs/원가·회수 시간	후보 생성·실패 평가·재학습 포함
Serving	p50/p95/p99, throughput, SLO goodput	concurrency·query mix·warm/cold 정의
물리 비용	peak residency, bytes, refresh/GC writes	동일 HBM/HBF package·전력 budget
복구 / 권한	duplicate/stale commit, rollback, policy violation	fault injection·권한 변경·감사 경계

Baseline은 의도적으로 강하게

같은 데이터와 budget으로 full context, hybrid retrieval+reranker, write-time summary, JIT curation, latent/LoRA, skill/program을 비교한다. 새로운 시스템에 더 많은 토큰·훈련·검증을 몰아주고 우수하다고 결론내리지 않는다.

평균과 신뢰 구간을 함께

task와 user 단위 paired evaluation 및 여러 실행을 수행한다. 성공 사례를 모은 pass@k와, 모든 반복에서 성공해야 하는 운영 신뢰도는 다르다. 백그라운드 update 후 평균 향상뿐 아니라 tail failure와 regression도 본다.

금지할 비교

논문 A의 LoCoMo 점수, 논문 B의 다른 model/reader 점수, 논문 C의 tool success를 한 종합 leaderboard로 묶지 않는다. 본 보고서의 숫자는 의도적으로 같은 연구 내부의 비교로 제한했다.

08 / RESEARCH AGENDA

연구 로드맵: 바꾸기 전에 반증 조건을 정한다

데이터 계약 → 강한 baseline → 승격 효용 → 물리 배치 순서로 위험을 줄인다.



Advance only after the previous contract passes its falsification test.

그림 31. 구현 순서 제안. 날짜 약속이 아닌 검증 gate 중심의 실행 로드맵이다. 본 보고서 실행 계획.

표 14. 우선순위 실험과 중단 조건

실험	동일 예산에서 비교	반증 / 중단 조건
E1 Consolidation timing	write summary vs raw+index vs JIT vs hybrid	JIT overhead를 포함하면 utility/cost 이득이 사라짐.
E2 Parametric promotion	retrieval vs skill vs latent vs LoRA	추가 훈련·검증·갱신 비용이 미래 reuse saving보다 큼.
E3 Dreaming curriculum	uniform replay vs failure-targeted vs policy search	OOD 전이 없이 selection task만 향상.
E4 Validity / rollback	TTL-only vs version+dependency+publish gate	신규 사실 반영 지연·false block이 안전 이득을 상쇄.
E5 HBF placement	stable-only vs compiled KV vs dynamic full-HBF	같은 package/thermal budget에서 p99/SLO가 baseline보다 악화.
E6 Influence policy	retrieval 고정, activation/influence만 변경	calibration 효과가 reader prompt 변경으로 동일하게 재현됨.

가장 먼저 얻어야 할 결과

E1과 E4를 CPU/기존 GPU 환경에서 끝내고, 신뢰 가능한 source-to-artifact trace를 확보한다. 그 다음 E2/E3의 학습 효용과 비용을 재현한 뒤, E5에서 physical placement를 바꾼다. 아직 유용성이 입증되지 않은 artifact를 빠르게 옮기는 하드웨어 최적화부터 시작하지 않는다.

08 / MEASUREMENT BLUEPRINT

DSE를 위한 계측 명세

Logical intelligence metric과 physical system metric을 join할 수 있어야 한다.

표 15. DSE parameter와 response 변수

독립 변수	권장 sweep 축	기대 관측
Read behavior	sparsity, burst size, hot set, reuse distance	effective BW, read amplification, prefetch hit, stall
Update behavior	interval, bytes/update, delta/full rewrite	write coalescing, GC/refresh 포함 physical bytes
Agent mix	concurrency, tool-gap 분포, returning sessions	residency survival, TTFT/ITL, SLO goodput
Learning mix	replay budget, LoRA rank, candidate count	validation yield, transfer gain, amortization horizon
Device budget	HBM 용량/BW, HBF BW/latency, temperature	Pareto frontier: quality·latency·cost·endurance
Control policy	promotion threshold, abstain, invalidate fan-out	unsafe reuse, rebuild cost, rejected beneficial updates

실험 trace의 최소 레코드

⟨task, object, source-version, artifact-version, phase, tier, bytes, time, outcome⟩

여기에 model/tokenizer/decoder revision, tool snapshot, eval dataset revision을 manifest로 연결한다.

같은 useful work를 비교한다

한 설계가 낮은 품질의 답을 더 빨리 내는 경우 throughput 만 비교하면 잘못된 결론에 이른다. acceptance threshold 를 통과한 task의 goodput을 우선 측정한다. 평균 latency 뿐 아니라 synchronous revalidation이 일으키는 tail도 기록한다.

한 가지 “효율 점수”로 뭉개지 않는다

정확도와 비용의 단위가 다르므로 임의 가중치를 붙인 단일 점수만 발표하지 않는다. 먼저 Pareto frontier를 공개하고, 그 위에서 workload별 SLO·전력·endurance 조건에 맞는 점을 선택한다.

수명 비용의 추가 경계

HBF endurance 분석에서는 host write와 NAND program bytes를 분리한다. refresh가 버전 교체·GC와 겹칠 때 두 번 세지 않도록 simulator accounting을 점검한다. logical commit count만으로 device lifetime을 추정하지 않는다. [46], [41]

09 / INDUSTRY & COMMUNITY

논문 밖에서도 control plane이 구체화된다

API·운영 기능의 성숙과 신경망 학습 메커니즘의 성숙을 분리해 읽는다.

제품에서는 memory가 독립 기능을 넘어 background worker와 framework primitive로 내려오고 있다. 이는 연구 메커니즘의 실증과는 다른 종류의 근거다. 제품 문서는 제공되는 API와 수명 관리 경계를 보여주지만, 특정 memory algorithm의 일반적 우월성을 입증하지는 않는다.

표 16. 확인한 제품 신호

자료 / 시점	공식적으로 설명된 동작	보고서 해석
Vercel eve 2026-09-09 [52]	named slot에 provider와 scope를 지정. turn 전 관련 memory를 context에 추가. private Blob 기반 지속 저장.	memory isolation과 backend 교체가 framework contract가 된다.
Grok Build 2026-09-16 [53]	완료 turn에서 durable note를 capture. /dream으로 topic Markdown에 병합; 현재 대화 지시 우선.	background semantic organization. 정책 탐색형 dreaming과 구별.
Hindsight Mental Models 2026-10-06 확인 [54]	standing answer를 background 또는 schedule로 갱신. freshness·source scope·version history 제공.	read-time 합성 비용을 사전 계산된 view로 옮기는 제품 패턴.
Sandisk / SK hynix 2026-08-03 [47]	OCP HBF specification 공개와 ecosystem 협력 발표.	capacity tier가 표준화 대상으로 구체화. 실측 양산 성능은 별도.

Community는 문제를 찾는 데 사용한다

직접 확인한 r/LLMDevs 게시글은 chat log와 embedding만으로는 “flat recall”에 머문다는 문제를 제기하며 cross-session state를 어떻게 다루는지 묻는다. 이는 설계 질문의 현장성을 보여주는 사례일 뿐, 대표 표본이나 비교 실험은 아니다. [55]

산업 전망의 중심

가장 성숙한 단기 기회는 source/version/scope를 가진 durable store와 background materialized view다. 장기적 차별화는 그 위에서 어떤 상태를 학습하고 어떤 artifact로 compile할지 결정하는 정책에 있다. 단기 persistence와 장기 self-improvement를 같은 maturity로 평가하지 않는다.

09 / SOURCE AUDIT

출처 감사와 정정 기록

잘못된 링크에 더 좋은 디자인을 입히지 않는다. publication-quality의 출발점은 출처 계약이다.

이전 GitHub 인덱스에는 일부 논문의 arXiv 주소가 제목과 일치하지 않거나 이번 대조에서 확인되지 않는 문제가 있었다. 아래는 PDF에서 바로잡은 항목이다. 링크 실패만으로 논문이 존재하지 않는다고 단정하지 않았고, 원문 제목 검색과 올바른 페이지 대조가 된 경우에만 정정했다.

표 17. 이번 판에서 바로잡은 식별자

연구	이전 인덱스 ID	이 PDF의 확인된 ID
Hindsight Memory-PRM	2608.31254	2608.29605 [20]
KV-streams	2609.35893	2609.35750 [11]
HBFSim	2609.09902	2609.09800 [46]
Mem++	2609.39246	2610.02002 [15]
Schema	2609.39943	2609.39140 [34]
Beyond Memory / PoS	2609.40334	2610.01415 [35]
ActiveSaddler	2609.40515	2610.00906 [32]
EfficientAgent	2609.34901	2609.33762 [37]

주장의 강도도 정정했다

slow CMS = HBF라는 표현은 조건부 placement 가설로 바꿨다. HBF write endurance를 commit 횟수로 근사한 서술도 physical bytes·refresh·GC를 포함하도록 수정했다. 두 변경은 문헌의 실제 결과와 우리의 추론을 분리하기 위한 것이다.

숫자와 버전의 해석

MemCodex의 10.1% 상대 향상은 10.1%p와 다르다. Mem++의 strongest memory baseline 대비 향상과 RAG 대비 향상도 구별했다. HBFSim 20.8×는 simulator 실행 속도이며 inference speedup으로 표기하지 않았다. [33], [15], [46]

발행 범위

정정은 이 PDF와 동봉한 reference/source package에 반영했다. 이전 Markdown 파일이 자동으로 교체되었다고 가정하지 않는다. 서지 검토 범위와 pinned version은 참고문헌 대장에 남겼다.

09 / EXTENDED TRACKING REGISTER A

보존하되 근거로 승격하지 않은 자료

핵심 보고서의 55개 출처와, 이전 주간 기록의 추가 후보를 분리한다.

아래 항목은 이전 주간 조사에서 언급한 추가 자료다. 이번 판에서는 원문 방법·수치·버전을 충분히 재검증하지 않아 정량 결론의 근거에서 제외했다. 누락을 숨기지 않고 후속 조사 queue로 보존한다. 제목은 이전 기록의 표기를 사용하며 별도 검증 없이 링크나 성과 수치를 새로 부여하지 않는다.

표 18. 이전 조사 보존 목록 A

주제	기록된 자료명
Episodic / structure	PolyMemDB; MemForest; Activity Frames; Jev-Mem; Persistent Discovery Context; Selective Forgetting
Compression / transfer	Agent Memory Distillation; Explicit, Not Longer; Break It Down, Pass It On; StateComp; GEM
Credit / influence	RoMeRL; MemTrapBench; Controlled Memory Interference; MeClear; TRACER; Representational Empowerment
Validity / admission	AuthMem-Bench; Endogenous Authorization Laundering; Memory Trust Gap; Grounding Agent Memory; Typed Intention Store
Lifecycle / transaction	CommitKV; SuperLocalMemory 4.0; MemoryCPT; MemTX / MemTxn; TransMem
Routing / multi-agent	Gated-Memory Routing; HiPS; Chain-of-Experience; Working Set of a Coding Agent

재검증 체크리스트

① 제목과 arXiv ID가 일치하는지 ② v1과 최신판의 날짜·주장이 같은지 ③ 수치가 absolute/relative/%p 중 무엇인지 ④ model·reader·harness가 통제됐는지 ⑤ validation budget까지 비용에 들어갔는지 ⑥ 실제 구현, simulator, position paper 중 무엇인지 확인한다.

왜 모두 같은 신뢰도로 인용하지 않는가?

총망라된 자료집은 모든 항목을 사실로 승격하는 문서가 아니다. 검증된 근거와 아직 검증되지 않은 기록을 함께 보존하되, 서로 다른 지위를 부여하는 것이 장기적으로 재사용 가능한 리서치 메모리다.

09 / EXTENDED TRACKING REGISTER B

후속 검증이 필요한 연결 연구

기존 조사 범위는 유지하되, 정확성이 확인되지 않은 수치를 다시 발행하지 않는다.

표 19. 이전 조사 보존 목록 B

주제	기록된 자료명
Neural / parametric	TMEM; PEAM; COVE; UniMem; SelfMem; LiveMem; Language Models Need Sleep
Dream / self-improvement	Discovery by Dreaming; PILOT in the Loop; RSIAgent; Meta ⁿ ; AREX-2; On the Fragility of Self-Improving Agents
Runtime / agents	FMOS position paper; Cognitive Extensions for Dual-Process Language Agents; Auditing Harness Tampering
KV / serving	AgentKV; PReCache; HeadWiseKV; SGD-KV; MILO; AgentZip
HW / evaluation	Hardware-Managed Heterogeneous HBM and Flash / HMA; Thinkingbox; Agent Memory Challenge Cycle 2
Industry / practitioner	MongoDB Agent Memory Inside the Harness; PolarDB-X Agentic Memory; Heavybit LMCache interview; PLUR restore/rollback runbooks; 개별 weekly Reddit threads

이 보고서에서 대체 확인한 근거

persistent neural state는 TTT·Titans·Memory Caching·RPMem으로, JIT와 non-destructive write는 JitMem·Mem++로, causal credit은 Hindsight Memory·PRM·Credit Without Ground Truth·DRSR로 다뤘다. systems/HBF는 AgentSysBench·EfficientAgent·ReCache·ATTUNER·FLINT·HBFlex·HBFSim 등 확인한 자료에 근거했다.

다음 판에서 우선 볼 연결

PReCache와 AgentKV는 multi-LoRA 공유와 phase-aware state를 보완할 수 있다. Grounding Agent Memory는 외부 세계와의 재검증을, MeClear는 query-scoped suppression을 보완할 수 있다. 이 가능성은 연구 후보의 동기이며, 이번 판에서 해당 논문의 세부 성과를 확정하는 것은 아니다.

자료 수와 연구 품질은 같은 지표가 아니다

같은 source를 여러 번 요약해도 독립 evidence는 하나다. 서로 다른 글이 동일 실험을 재인용한 경우에도 evidence count를 늘리지 않아야 한다. 이 원칙은 리서치 보고서와 agent memory 모두에 적용된다.

10 / GLOSSARY & CONCLUSION

용어와 최종 명제

같은 단어로 서로 다른 계층을 부르지 않으면, 실험 가설도 더 명확해진다.

표 20. 이 보고서의 용어 계약

용어	의미 / 구분
Sleep-time compute	미래 질의를 위해 사전에 수행하는 계산. 본문에서는 학습·precompute·maintenance lane을 구분한다.
CMS	Nested Learning 원문의 Continuum Memory System. 외부 multi-tier DB와 동일한 구현이 아니다.
Canonical state	버전·출처 계약 아래 유지하는 기준 상태. 항상 진실이거나 불변이라는 뜻은 아니다.
Materialization	canonical/latent state에서 현재 실행에 필요한 prompt/KV/adapter/program을 만드는 과정.
Promotion	일회 경험이나 후보를 더 넓게 재사용되는 artifact로 승인하는 과정. 물리 이동과 별도.
Invalidation	객체나 파생 artifact의 사용 자격을 무효화. 원본 삭제나 byte rewrite와 다르다.
Rollback / compensation	이전 generation 복귀와 외부 side effect 상쇄는 다른 연산. fact-level unlearning과도 다르다.
Fast weights / LoRA	빠르게 바뀌는 parameter state와 저랭크 parameterization. LoRA라는 형식 자체가 학습 방식을 정하지 않는다.
HBF / HBM	flash 기반 capacity tier와 높은 대역폭의 DRAM 계층. 이름의 bandwidth만으로 read/write/thermal 특성을 같게 보지 않는다.
Causal utility	명시한 개입 조건에서 with/without 또는 alternate-action 결과의 차이. proxy의 범위를 기록한다.

최종 연구 명제

향후 AI memory의 핵심은 “더 많은 경험”이 아니라, **근거가 남아 있고, 현재도 유효하며, 필요한 실행 형태로 바꿀 수 있고, 미래 비용을 실제로 줄이는 상태**다. 이 명제는 연구 방향이다. 구현의 가치는 같은 예산의 강한 baseline과 비교해 결정된다.

REFERENCES / SOURCE REGISTER

주석형 참고문헌 01

제출일·갱신일·검토 범위를 기록했다. 제목과 원문 링크를 대조했으며, 열람 범위를 전체 본문 검토나 독립 재현으로 과장하지 않았다.

[01] Sleep-time Compute: Beyond Inference Scaling at Test-time

Kevin Lin et al. · 2025-04-17 · P · PREPRINT

질문 이전 compute를 여러 미래 query에 상각. query 예측 가능성에 의존.

검토: 초록·본문 선택 절·PDF 도판 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2504.13171>

[02] Sleep-time compute

Letta · 2025-04-21 · D · OFFICIAL

stateful agent에서 background memory processing을 설명한다.

검토: 공식 연구 블로그 · 확인 2026-10-06

<https://www.letta.com/blog/sleep-time-compute/>

[03] Nested Learning: The Illusion of Deep Learning Architectures

Ali Behrouz et al. · 2025-12-31 · P · PREPRINT

CMS는 서로 다른 갱신 주기의 MLP 계열. HOPE는 self-modifying Titans와 결합. 외부 DB 계층과 구별.

검토: §7 - 8, Eq.70 - 71·94 - 97, PDF pp.27·33 및 초록 · 확인 2026-10-06

<https://arxiv.org/abs/2512.24695>

[04] Introducing Nested Learning: A new ML paradigm for continual learning

Google Research · 2025 (블로그) · D · OFFICIAL

원 논문의 개념 설명. 학회 공개와 arXiv 등록일은 별도.

검토: 공식 연구 블로그 · 확인 2026-10-06

<https://research.google/blog/introducing-nested-learning-a-new-ml-paradigm-for-continual-learning/>

[05] Memory Caching: RNNs with Growing Memory

Ali Behrouz et al. · 2026-02-27 · P · PREPRINT

recurrent memory checkpoint를 확장해 고정 상태의 용량 한계를 완화한다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2602.24281>

[06] Online Neural Space Time Memory for Dynamic Novel View Synthesis

Baback Elmieh et al. · 2026-07-16 · P · PREPRINT

메모리 갱신과 적용 빈도를 분리. 영상 실험을 언어 agent 성능으로 일반화하지 않는다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2607.15271>

[07] Titans: Learning to Memorize at Test Time

Ali Behrouz, Peilin Zhong, Vahab Mirrokni · 2024-12-31 · P · PREPRINT

attention과 neural long-term memory를 결합. 출판 ID는 2501이나 최초 제출은 2024-12-31.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2501.00663>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 02

[08] Learning to (Learn at Test Time): RNNs with Expressive Hidden States

Yu Sun et al. · 2024-07-05 · 갱신 2025-08-31 v4 · P · PREPRINT

hidden state 자체를 학습 모델로 보고 test sequence에서 self-supervised update. TTT-MLP의 I/O 병목 명시.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2407.04620>

[09] RPMem: Learning Long-Term Recurrent Parametric Memory Across Sessions for LLM Agents

Fanyu Zhao et al. · 2026-09-20 · 갱신 2026-09-22 v2 · P · PREPRINT

recurrent latent memory와 backbone-specific LoRA decoder 분리. inference에서 모듈 weights는 frozen, 상태는 갱신.

검토: 초록·방법/훈련 프로토콜·PDF 도식 선택 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.23466v2>

[10] Infinite-Parameter LLMs: Generating and Adapting Weights from Live Data

Jinli Hu et al. · 2026-09-16 · 갱신 2026-09-21 v2 · P · PREPRINT

latent belief로 low-rank weights를 생성. 제목은 무한 정보 용량의 증명이 아니다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.18842>

[11] KV-streams for Efficient Compaction in Agentic Reinforcement Learning

Emiliano Penalzoa et al. · 2026-09-28 · 갱신 2026-09-29 v2 · P · PREPRINT

compaction 후 retained KV를 유지. persistent KV가 recurrent 정보 경로가 될 수 있다.

검토: 초록·HTML 선택 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.35750>

[12] MemoryAthena: Adaptive Routing over Latent and Generated Memories

2026-09-22 · 갱신 2026-09-26 v2 · P · PREPRINT

direct retrieval과 두 generated path를 선택. router만 학습; hallucinated memory와 관측을 구별해야 한다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.25853>

[13] MemCalib: Benchmarking and Optimizing Memory Use in LLM Agents

Ruike Cao et al. · 2026-09-21 · 갱신 2026-09-22 v2 · P · PREPRINT

over-use와 under-use를 별도 측정하고 influence를 보정한다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.24259>

[14] Just-in-Time Memory: Learning to Curate Task-Adaptive Memory for LLM Agents

Yefan Zhou et al. · 2026-09-23 · P · PREPRINT

raw trajectory를 보존하고 query 도착 후 memory payload를 구성. online curation 비용 포함 필요.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.27334>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 03

[15] Mem++: Non-Destructive Memory for Long-Term Organizational LLM Agents

Ahmad Yehia et al. · 2026-10-01 · P · PREPRINT

원문·작성자·날짜를 보존. strongest memory baseline 대비와 RAG 대비 이득을 구별.

검토: 초록·HTML 결과 선택 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2610.02002>

[16] Correct Now, Insufficient Later: Auditing Update Sufficiency in Context Compression

Guangzhe Zhang · 2026-09-17 · P · PREPRINT

현재 답변 충분성과 미래 갱신 충분성의 차이. 24 paired-history synthetic pilot; 자연 과제 검증 없음.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.20045>

[17] DRSR: Learning Set-Level Deletion Risk for Efficient Long-Horizon Agents

2026-09-23 · P · PREPRINT

집합 단위 삭제 후 next-output likelihood를 supervision으로 사용. 환경 outcome의 완전한 인과 효과는 아니다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.27276>

[18] Can Agent Memory Systems Track Evolving State?

Xinyi Fan et al. · 2026-08-20 · P · PREPRINT

234 scenarios, supersession·dependency를 통해 current-state tracking 평가.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.19652>

[19] Remember, Verify, or Ask? Cross-Family Evaluation of Memory Commitment in LLM Agents

2026-08-20 · P · PREPRINT

persist / local / verify / clarify의 admission 선택을 구분.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.19564>

[20] Hindsight Memory-PRM: Supervising Memory Management with Auditable Hindsight Credit

Haoxuan Jia et al. · 2026-08-30 · P · PREPRINT

retrieval/citation과 deletion-and-reanswer로 entry presence credit을 추정; version chain을 따라 proxy reward 부여.

검토: 초록·HTML 결론 · 확인 2026-10-06

<https://arxiv.org/abs/2608.29605>

[21] Credit Without Ground Truth: Auditing Step-Level Credit Assignment in LLM Agents Against Executed Replay

Haiyue Zhang · 2026-08-20 · 갱신 2026-08-25 v2 · P · PREPRINT

ALFWorld에서 replay-conditional contribution을 audit. correctness label과 contribution은 다르다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.19760v2>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 04

[22] Fresh Memory, Stale Plans: Dependency-Scoped Validation for Distributed LLM-Agent Memory

Evan Chen, Shiqiang Wang, Christopher G. Brinton · 2026-09-03 · P · PREPRINT

계획이 참조한 record version을 action 직전 검사. 이후 제목 변경판과 수치를 혼합하지 않음.

검토: v1 초록·HTML; v1에 고정 · 확인 2026-10-06

<https://arxiv.org/abs/2609.03340v1>

[23] Invalidation Contracts for Cross-Episode Agent Memory

Michael Wu, Arquimedes Canedo · 2026-08-31 · P · PREPRINT

version stamps와 cacheability hints. validity는 protocol, compliance는 planner 특성.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.00243>

[24] LLM Agents Can Easily Tamper With Their Own Traces

Jeremy Qin et al. · 2026-09-24 · P · PREPRINT

agent가 제어할 수 없는 독립 logging 경계가 필요. 본 보고서는 공격 절차를 제공하지 않는다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.30266>

[25] When Malicious Instructions Persist: Persistent Memory Poisoning Attack on Harness-Based Agents

Shuhuai Huang, Jingfeng Zhang, Hong Jia · 2026-09-12 · P · PREPRINT

cross-session memory poisoning 위험. admission quarantine과 실행 권한 분리의 근거.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.13889>

[26] Where Does Exactly-Once Live? Model, Harness, and Tool-Contract Effects on Duplicate Side Effects in LLM Agents

Jiapeng Li · 2026-09-24 · P · PREPRINT

25,930 episodes. read-back verification만으로 late commit ambiguity 해소 불가; idempotency contract 필요.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.29095>

[27] Dream-RSI: Recursive Self-Improvement through Evolving Worlds

Tong Zheng et al. · 2026-09-14 · P · PREPRINT

실제로 탐색된 discovery tree를 replay world로 사용. 지원되지 않는 반사실 경로까지 검증한 것은 아니다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.14858>

[28] WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution

Liyan Tang et al. · 2026-08-27 · P · PREPRINT

episode / persistent wiki / executable skill을 분리하고 공동 진화.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.27454>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 05

[29] Recursive Experiential-Working Memory Evolution for Long-Horizon Agent Harnesses

Zhaochen Yu et al. · 2026-08-25 · P · PREPRINT

현재 working state가 skill 호출을 안내; 고정 meta-agent가 validation-gated 국소 변경.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.24876>

[30] Recursive self-improvement of AI research agents

Dhruv Srikanth et al. · 2026-09-22 · P · PREPRINT

AIDE²가 8일 run에서 7개 successive improvement를 발견. hidden evaluation 및 held-out 전이 검증.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.26457>

[31] RRSI: Regularized Recursive Self-Improvement of Agent Harnesses

Peng Xia et al. · 2026-09-21 · 갱신 2026-09-23 v2 · P · PREPRINT

edit budget annealing·critic·pruner로 harness evolution 과적합을 줄인다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.24972v2>

[32] ActiveSaddler: Automated Curriculum Learning for Agent Harness Optimization

Sungho Park et al. · 2026-10-01 · P · PREPRINT

non-stationary bandit으로 failure-pattern curriculum을 최적화. background budget 선택에 연결 가능.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2610.00906>

[33] MemCodex: Self-Programming Hierarchical Memory for Language Agents

Xiaoqiang Wang, Bang Liu · 2026-09-30 · P · PREPRINT

construction/index/retrieval/routing을 executable memory program으로 진화. Work in progress 명시.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.39765>

[34] Schema: Discovering Unknown Environments via Agentic Program Induction

Guanning Zeng et al. · 2026-09-30 · P · PREPRINT

환경 상호작용을 실행 가능한 predictive program으로 구성. 특정 게임/환경 결과.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.39140>

[35] Beyond Memory: Harnessing Long-Horizon Agents with Explicit Belief States

Yu Luo et al. · 2026-10-01 · P · PREPRINT

world estimate와 unresolved requirements를 active belief state에 유지하고 trapping 감지.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2610.01415>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 06

[36] ATTUNER: Recomputation-Free KV Cache Reuse via Query-Side Adaptation

Xinghao Chen et al. · 2026-09-29 · P · PREPRINT

frozen artifact KV는 유지하고 query projection low-rank adapter만 학습. HBF 실험은 아님.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.36722>

[37] EfficientAgent: What Makes KV Cache Offloading Work for Concurrent Agents?

Kunming Shao et al. · 2026-09-27 · P · PREPRINT

offload tier를 agent-pool reuse working set과 stack distance로 평가.

검토: 초록·HTML 선택 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.33762>

[38] UNISON: A Co-Designed Near-Memory Scheduler of Session KV Residency for LLM Agents

Fan He, Yan Li, Xiaoyang Zeng · 2026-09-09 · P · PREPRINT

tool return gap을 활용한 phase-aware residency와 migration.

검토: 초록·HTML 선택 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.09643>

[39] From LLM Inference to Agentic Workloads: A Systems Characterization

Chaokun Chang et al. · 2026-08-15 · P · PREPRINT

AgentSysBench, 10개 응용에서 LLM·tool·sandbox·idle state를 분해.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.15127>

[40] FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration

Geraldo F. Oliveira et al. · 2026-08-25 · P · PREPRINT

burst buffer·phantom-plane refresh·read-only FTL. 임의 쓰기 지원을 가정하면 안 됨.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.25062>

[41] HBFlex: A Flexible Memory System for Bridging Fine-Grained LLM States and Coarse-Grained HBF Parallel Execution

Shuzhang Zhong et al. · 2026-09-16 · P · PREPRINT

full-HBF weights+KV, plane balance·write aggregation·lifetime packing. trace-driven simulation.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.18675>

[42] Trillion-Parameter MoE in a Box: Decoupling Memory Provisioning with High-Bandwidth Flash

Pengfei Xia et al. · 2026-09-14 · P · PREPRINT

HBF weights와 DRAM runtime state의 capacity/BW를 분리. low-concurrency trace 조건.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.15636>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 07

[43] HBF Sucks? A Full-Stack Characterization of HBF for KV-Centric LLM Serving

2026-08-12 · 갱신 2026-09-14 v4 · P · PREPRINT

transient KV를 SSD 대신 HBF에 단순 offload할 때 악화. TokenSim 기반 시스템 분석.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.11668v4>**[44] Potential Applications of HBF in LLM Serving Systems**

2026-08-13 · 갱신 2026-08-14 v2 · P · PREPRINT

model-state capacity와 expert/model residency 확장. HBM execution path 보존 조건.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.13127>**[45] Exploring High-Bandwidth Flash for Modern LLM Inference**

Dowon Son et al. · 2026-08-14 · P · JOURNAL

IEEE CAL, DOI 10.1109/LCA.2026.3705817. read BW·endurance 조건이 중요.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.13868>**[46] HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution**

2026-09-09 · P · PREPRINT

real GPU execution에 HBF timing/thermal/refresh model 삽입. 20.8×는 simulation speedup.

검토: v1 초록·§4·Figure 1·PDF p.8 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.09800v1>**[47] Sandisk and SK hynix Advance Global Standardization of HBF**

Sandisk / SK hynix · 2026-08-03 · D · OFFICIAL

OCP specification 공개 및 생태계 협력. 양산 제품 성능 측정과 구별.

검토: 공식 보도자료 · 확인 2026-10-06

<https://www.sandisk.com/company/newsroom/press-releases/2026/2026-08-03-Sandisk-and-sk-hynix-advance-global-standardization-of-hbf>**[48] DolphinBench: Mapping the Pareto Frontier of Agent Memory**

Soumil Rath, Deshraj Yadav, Taranjeet Singh · 2026-09-21 · 갱신 2026-09-22 v2 · P · PREPRINT

3 personas×200 tasks, history with/without 조건; accuracy 외 total cost·latency 요구.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.24971v2>**[49] Memory Attention**

Jiale Kang · 2026-09-23 · P · PREPRINT

token memory와 contextual key로 value를 구성. CPU offload/prefetch 가능성, 추가 parameter 조건.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2609.28399>

REFERENCES / SOURCE REGISTER

주석형 참고문헌 08

[50] ReCache: Efficient KV Cache Reuse and Compression for Tool-Augmented LLM Agents

Yichu Fang et al. · 2026-08-20 · P · PREPRINT

resource-local KV와 구조/의미 pruning. Inv-F1 82.3 vs82.4; TTFT 3.655×

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.19662>

[51] MESA: Task-Adaptive Multi-Structure Evidence Selection for Long-Horizon Agent Memory

Beidi Zhao et al. · 2026-08-10 · P · PREPRINT

5개 structural view 중 query-specific subset 선택. 모든 memory structure를 읽지 않는다.

검토: 초록·서지 확인 · 확인 2026-10-06

<https://arxiv.org/abs/2608.10108>

[52] Persistent memory for eve agents

Andrew Barba / Vercel · 2026-09-09 · D · OFFICIAL

slot/provider/scope를 framework primitive로 제공. 자체 학술 비교실험은 아님.

검토: 공식 changelog · 확인 2026-10-06

<https://vercel.com/changelog/persistent-memory-for-eve-agents>

[53] Memory in Grok Build

SpaceXAI · 2026-09-16 · D · OFFICIAL

completed turn capture, topic Markdown, /dream background organization. neural policy dreaming과 구별.

검토: 공식 제품 문서 · 확인 2026-10-06

<https://x.ai/news/grok-build-memory>

[54] Mental Models

Vectorize / Hindsight · 날짜 미표시 · D · OFFICIAL

standing answer를 background 갱신: source scope·freshness·version history. 제품 기능 문서.

검토: 2026-10-06 문서 확인 · 확인 2026-10-06

<https://hindsight.vectorize.io/developer/mental-models>

[55] How are you handling persistent memory in LLM apps?

r/LLMDevs 커뮤니티 · 날짜 미확인 · C · COMMUNITY

chat log+embedding과 지속 state의 차이에 관한 현장 질문. 성능·채택률 근거로 사용하지 않음.

검토: 게시물 본문 · 확인 2026-10-06

https://www.reddit.com/r/LLMDevs/comments/1r35hlc/how_are_you_handling_persistent_memory_in_llm_apps/

PRODUCTION NOTES

매체 출처와 발행 정보

원본 그림의 의미와 보고서의 해석을 구별할 수 있도록 제작 이력을 남긴다.

본문의 모든 diagram과 chart는 이 보고서를 위해 작성한 SVG 벡터 원본이다. 원문 figure를 이미지 파일로 복사하지 않았다. 논문 메커니즘을 설명하는 독립 도식과 수치의 재시각화는 각 caption에 출처·변환 유형을 표시했다. 표지 역시 개념을 추상화한 벡터 디자인으로, 뇌 구조나 실제 memory device의 현미경 사진이 아니다.

표 21. 미디어 제작·해석 계약

구분	제작 / 해석
메커니즘 도식	논문 또는 다수 문헌의 개념을 다시 설계. 실제 구현 상세를 생략한 경우 caption에 명시.
데이터 그래프	JitMem·LIMBO 등 동일 연구 내부의 확인된 수치만 사용. 축·단위·비교 조건 명시.
시스템 설계도	본 보고서의 가설. 기존 paper나 Melete/HAT가 동일하게 구현됐다는 의미가 아님.
비용 예시	손익 분기점 그래프의 비용 단위와 수치는 가상. 실측 결과와 시각적으로 구별.
수치 검증 한계	모든 benchmark는 저자 보고값이며 실행 재현 없음. 보고서 자체의 benchmark dataset은 없음.

재현 가능한 편집 원본

동봉 패키지에는 PDF, HTML 원본, CSS, SVG assets, reference JSON/CSV, source-audit notes, figure/table manifest, 생성 Python 코드를 포함한다. 사용한 글꼴 파일 자체는 배포하지 않는다. 빌드는 한국어 글꼴과 WeasyPrint가 설치된 환경을 전제로 한다.

판권·발행 안내

문헌 저작권은 각 저자와 발행처에 있다. 본 보고서는 출처를 설명·비평·종합한 독립 연구 문서이며 Google, Sandisk, SK hynix, Vercel, Vectorize, xAI 또는 학회/학술지의 승인·공식 발행을 주장하지 않는다. 본 보고서의 기술 제안은 검증 전 가설이며 제품 보증이나 투자 의견이 아니다.

Version 2.0 · 2026-10-06

기존 Markdown 종합을 한국어 PDF로 재구성하고, 식별자·주장의 강도·단위·근거 범위를 정정했다. 마지막 검토일 이후 논문과 제품 문서는 바뀔 수 있다. 정정 시에는 reference ID와 figure/table 번호를 기준으로 추적한다.